IMPERIAL

Scaled High Temperature Test Reactor Cogeneration for Zero Emission Hybrid-Desalination with Independent Water Heat Rejection

Imperial High Temperature Gas Cooled Reactor 5 and Heat Rejection 5

Authored by:

[HTGR]
Muhammad Dani
Michal Krasowski
Qiji Zhang
Clayton Yeung

[HR]
Oscar Chan
Heidi Chong
Oliver Bush
Muhammad Abubakar
Luca Faillace

Imperial College London
March 2026

Contents

Notation

Variables and Symbols

$P(t)$	Normalised reactor power
$\rho(t)$	Reactivity
β	Total delayed neutron fraction
β_i	Delayed neutron fraction for precursor group i
Λ	Prompt neutron lifetime
λ_i	Decay constant for delayed neutron group i
$C_i(t)$	Delayed-neutron precursor concentration for group i
$T_f,\ T_m,\ T_c$	Fuel, moderator, and coolant temperature
$T_{\mathbf{in}}$	Coolant inlet temperature
$T_{\mathbf{sat}}$	Saturation temperature
T_s	Surface temperature
T_a	Ambient air temperature
ΔT	Temperature difference
$C_f,\ C_m,\ C_c$	Heat capacities of fuel, moderator, and coolant
C_p	Specific heat capacity at constant pressure
$h_{\mathbf{fm}}$	Heat transfer coefficient, fuel–moderator
$h_{\mathbf{fc}}$	Heat transfer coefficient, fuel–coolant
$h_{\mathbf{mc}}$	Heat transfer coefficient, moderator–coolant
U	Overall heat transfer coefficient
$Q_{\mathbf{fission}}$	Fission heat generation rate
$Q_{\mathbf{total}}$	Total rejected heat
$Q_{\mathbf{mod}}$	Air energy capacity per module
$Q_{\mathbf{eff}}$	Effective air energy capacity per module
$q''_{\mathbf{total}}$	Total heat flux
$q''_{\mathbf{rad}}$	Radiative heat flux
$q''_{\mathbf{solar}}$	Solar heat flux
$q''_{\mathbf{conv}}$	Convective heat flux
$\dot{m}$	Mass flow rate
$\dot{m}_{\mathbf{He}}$	Helium mass flow rate
$\dot{m}_{\mathbf{steam}}$	Steam mass flow rate
$\dot{m}_{\mathbf{air}}$	Air mass flow rate
$\dot{V}_{\mathbf{mod}}$	Volumetric airflow per module
$\dot{V}_{\mathbf{total}}$	Total volumetric airflow
$W_{\mathbf{helium\_turb}}$	Helium turbine work output
$W_{\mathbf{steam\_turb}}$	Steam turbine work output
$P_{\mathbf{fan}}$	Fan parasitic power
$\eta_{\mathbf{thermal}}$	Thermal efficiency
$\eta_{\mathbf{fan}}$	Fan efficiency
$\eta_{\mathbf{foam}}$	Porous foam fin efficiency
h	Specific enthalpy
$x_{\mathbf{in}}$	Inlet vapour quality

$A_{\textbf{eff}}$	Effective surface area
$A_{\textbf{foam}}$	Porous foam condensation area
$A_{\textbf{coil}}$	Air-cooler coil surface area
a_s	Specific surface area of porous foam
$V_{\textbf{foam}}$	Volume of porous foam
$k_{\textbf{eff}}$	Effective porous conductivity
$R_{\textbf{tot}}$	Total thermal resistance
$N_{\textbf{mod}}$	Number of air-cooler modules
$\Delta P_{\textbf{air}}$	Air-side pressure drop
$UA_{\textbf{req}}$	Required overall conductance
$u,\ v$	Velocity components in x- and y-directions
ν	Kinematic viscosity
p	Pressure

Acronyms and Abbreviations

HTTR	High-Temperature Engineering Test Reactor
HTGR	High-Temperature Gas-Cooled Reactor
MED	Multi-Effect Distillation
DEEP	Desalination Economic Evaluation Program
HRSG	Heat Recovery Steam Generator
MATSCF	Modelling and Analysis Tool for Steady Channel Flow
HR	Heat Rejection
HX	Heat Exchanger
GOR	Gain Output Ratio
EUF	Energy Utilisation Factor
LMTD	Log Mean Temperature Difference
IRR	Internal Rate of Return
CAPEX	Capital Expenditure
ODE	Ordinary Differential Equation
TVC	Thermal Vapour Compression

Part I: High Temperature Gas Reactor Applied to Hybrid Desalination

1 Introduction, Theory and Methodology

The global energy landscape is currently facing two simultaneous crises: rapidly rising energy demand and the urgent need for deep decarbonization. As traditional fossil-fuel-intensive processes are continuously being displaced, High-Temperature Gas-Cooled Reactors (HTGRs) have emerged as a promising technology. Their ability to produce high-grade process heat, reaching upwards of $950\,^{\circ}\mathrm{C}$ enables applications far beyond simple electricity generation. Our Project utilises a scaled High-Temperature Engineering Test Reactor (HTTR) architecture to address the supply of fresh water to a large population. By coupling this reactor with a $300\,\mathrm{MW}$ thermal capacity and a hybrid Multi-Effect Distillation (MED) complex, this project provides a zero-emission solution for two vital resources. The primary objective is to sustain a daily population of $80{,}000$ with high-purity freshwater while maintaining a net-zero carbon footprint by displacing gas-fired desalination and grid-based power.

1.1 Literature Review and Down Selection Process

The early stages of the project focused on defining a resilient framework for the carbon-free co-generation of high-purity water and grid-scale electricity. The primary aim is to establish an integrated energy station that addresses regional water scarcity through prioritising thermal stability, passive safety, and minimal carbon footprint over traditional fossil-fuel architectures. The down-selection to the $300\,\mathrm{MW}$ HTTR-Desalination baseline is preferred because it uniquely bridges the gap between high-enthalpy power cycles and low-grade thermal distillation. Unlike Pebble-Bed or Micro-HTGR designs, the HTTR variant provides a consistent $950\,^{\circ}\mathrm{C}$ outlet temperature, ensuring that after primary power extraction, the exhaust quality remains high enough to drive Multi-Effect Distillation (MED) at scale. This choice minimizes entropy production and maximises exergy efficiency.

Table 1: Down-selection process for preferred model configuration

Reactor Type	Key Parameters	Advantages	Limitations
Pebble-Bed HTGR	200–$300\,\mathrm{MW_t}$; 70–$80\,\mathrm{MW_e}$; $750\,^{\circ}\mathrm{C}$ outlet	Proven reactor architecture; Industrial scale output	Complex systems; Not optimised for MED desalination
Micro-HTGR	0.5–$20\,\mathrm{MW_t}$; 0.1–$5\,\mathrm{MW_e}$; $750\,^{\circ}\mathrm{C}$ outlet	Very passive safety; Portable/remote use	Very small output; Limited desalination capability
HTTR-Desalination	$300\,\mathrm{MW_t}$; 10–$20\,\mathrm{MW_e}$; $950\,^{\circ}\mathrm{C}$ outlet	Ideal for MED desalination; High stability and safety	Research-derived design

1.2 Key Metrics and Reasoning for HTTR-Desalination Complex

Table 2: Key metrics and target HTTR-Desalination values

Metric	HTTR-Desalination Values
Reactor Thermal Power	$300\,\mathrm{MW_{th}}$
MED GOR	10–12
Water Production	$7{,}341$–$15{,}256\,\mathrm{m^3/day}$ (theoretical)
Cooling Requirement	Very low
Electricity Output	30–$50\,\mathrm{MW_e}$

Table 3: Advantageous parameter reasoning for HTTR-Desalination

Reason Category	HTTR-Desalination Advantage
High-temperature output	$950\,^\circ\mathrm{C}$ design $\rightarrow$ supports high-efficiency MED/TVC
Right-sized thermal power	$300\,\mathrm{MW_{th}}$ matches a semi-industry scale desalination plant
Water production capability	9–$12\,\mathrm{k\,m^3/day}$ potable water reliably
Electricity cogeneration	30–$50\,\mathrm{MW_e}$ from small Rankine/H_2O cycle
Cooling method	Liquid metal and air-cooled condensers at high temperature; radiative cooling $< 200\,^\circ\mathrm{C}$
Safety	Strong passive characteristics; high heat capacity graphite
Community suitability	Small footprint; minimal marine impact; stable $24/7$ output
Independent heat rejection power	$148\,\mathrm{MW_{th}}$

1.3 Reactor Scaling

From our Kinetics Block, the Q_{nominal} that we manage to achieve is $300\,\mathrm{MW_{th}}$ and is normalised with $P(t) = 1$, which indicates a $300\,\mathrm{MW_{th}}$ energy transfer into the Thermal Block. The mass flow rate of helium in the original HTTR is 10.5–$12.3\,\mathrm{kg/s}$; hence, in our model, we are using $105\,\mathrm{kg/s}$.

1.4 Python Usage for Reactor Modelling in MATLAB and Simulink

To develop a dynamic and realistic Helium Coolant Loop across our Reactor and Power Extraction Cycles, the CoolProp function from Python was utilised in the MATLAB and Simulink directories, enabling us to transition our helium fluid to a Real Gas Model where C_p is a function of both temperature and pressure $C_p(T)$. This is particularly vital for our power extraction cycles as the helium coolant undergoes a massive temperature swing from the core inlet to the secondary heat exchangers. Moving away from the Constant C_p (ideal gas) assumption in our Simulink model and integrating CoolProp was a critical step for achieving high-fidelity results. This step is crucial before designing the model as Simulink did not contain helium properties for modelling despite having fluid properties for steam.

2 Methodology of Reactor Design and Modelling Analysis

2.1 Point Kinetics

The reactor kinetics and core thermal dynamics were first represented using a compact set of coupled ordinary differential equations (ODEs).

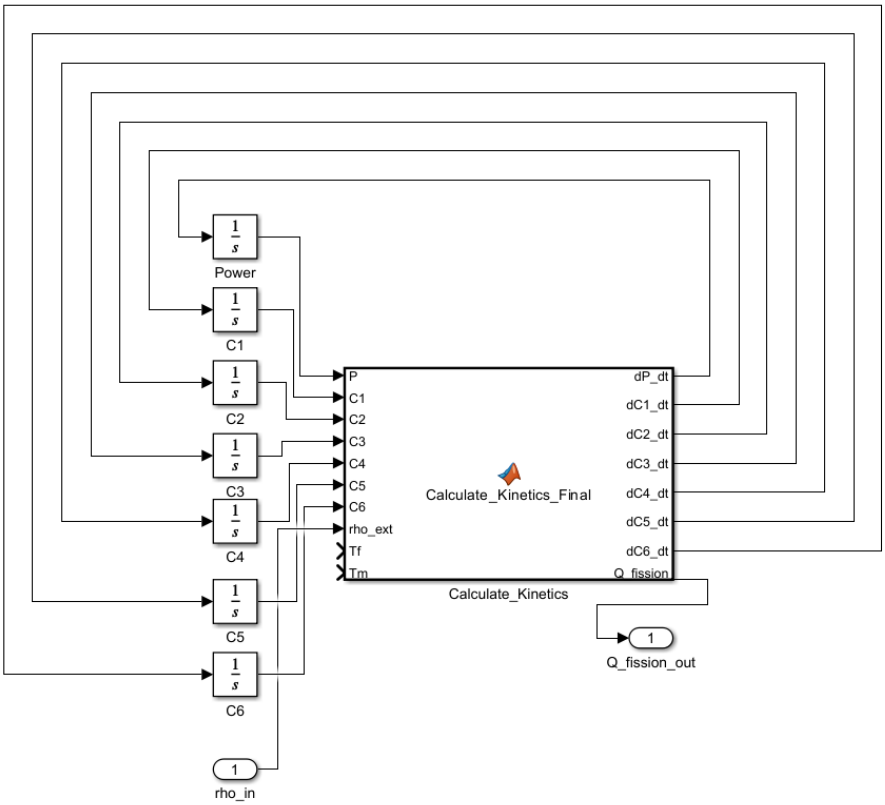


Figure 1: Point Kinetics block coded within MATLAB and implemented in Simulink

The neutronics (power generation) were modelled using the standard point-kinetics formulation with one ODE for the normalised reactor power $P(t)$ and six ODEs for the delayed-neutron precursor groups $C_i(t)$:

$$\frac{dP(t)}{dt} = \left(\frac{\rho(t) - \beta}{\Lambda}\right) P(t) + \sum_{i=1}^{6} \lambda_i C_i(t) \tag{2.1}$$

$$\frac{dC_i(t)}{dt} = \left(\frac{\beta_i}{\Lambda}\right) P(t) - \lambda_i C_i(t) \tag{2.2}$$

where Equation (2.1) is the reactor power equation and Equation (2.2) describes the precursor groups. In these equations, t is time (s), $P(t)$ is dimensionless power relative to nominal operation (so $P = 1$ corresponds to nominal power). These precursor inventories produce delayed neutrons and introduce a slower "memory" into the power response. The reactivity $\rho(t)$ quantifies departure from criticality, β is the total delayed neutron fraction, β_i is the fraction assigned to precursor group i, Λ is the prompt neutron lifetime, and λ_i are the decay constants for each delayed group. By explicitly modelling the decay constants and yields for each group, the system accurately simulates the memory effect essential for reactor control and safety analysis. This mathematical rigour allows the Simulink model to maintain stability even during significant reactivity perturbations, ensuring

that the power response $P(t)$ remains physically achievable and consistent with the established nuclear theory.

2.2 Thermal Core

The thermal core was represented by three lumped nodes: fuel (T_f), moderator (T_m), and coolant (T_c) with corresponding heat capacities C_f, C_m, and C_c. Heat transfer between nodes was captured using effective coefficients h_{fm}, h_{fc} and h_{mc}, and coolant heat removal was driven by the helium mass flow rate $\dot{m}$ and inlet temperature T_{in}. The governing energy balances were:

$$C_f \frac{dT_f}{dt} = Q_{\text{fission}} - h_{\text{fm}}(T_f - T_m) - h_{\text{fc}}(T_f - T_c) \tag{2.3}$$

$$C_m \frac{dT_m}{dt} = h_{\text{fm}}(T_f - T_m) - h_{\text{mc}}(T_m - T_c) \tag{2.4}$$

$$C_c \frac{dT_c}{dt} = h_{\text{fc}}(T_f - T_c) + h_{\text{mc}}(T_m - T_c) - 2\dot{m}C_p(T_c - T_{\text{in}}) \tag{2.5}$$

Implemented in Simulink, each ODE was integrated in time, and the kinetics and thermal subsystems were coupled by passing $Q_{\text{fission}}(t)$ into the thermal model while allowing temperatures to feed back into $\rho(t)$, enabling simulation of coupled power–temperature transients suitable for integration with the desalination plant model.

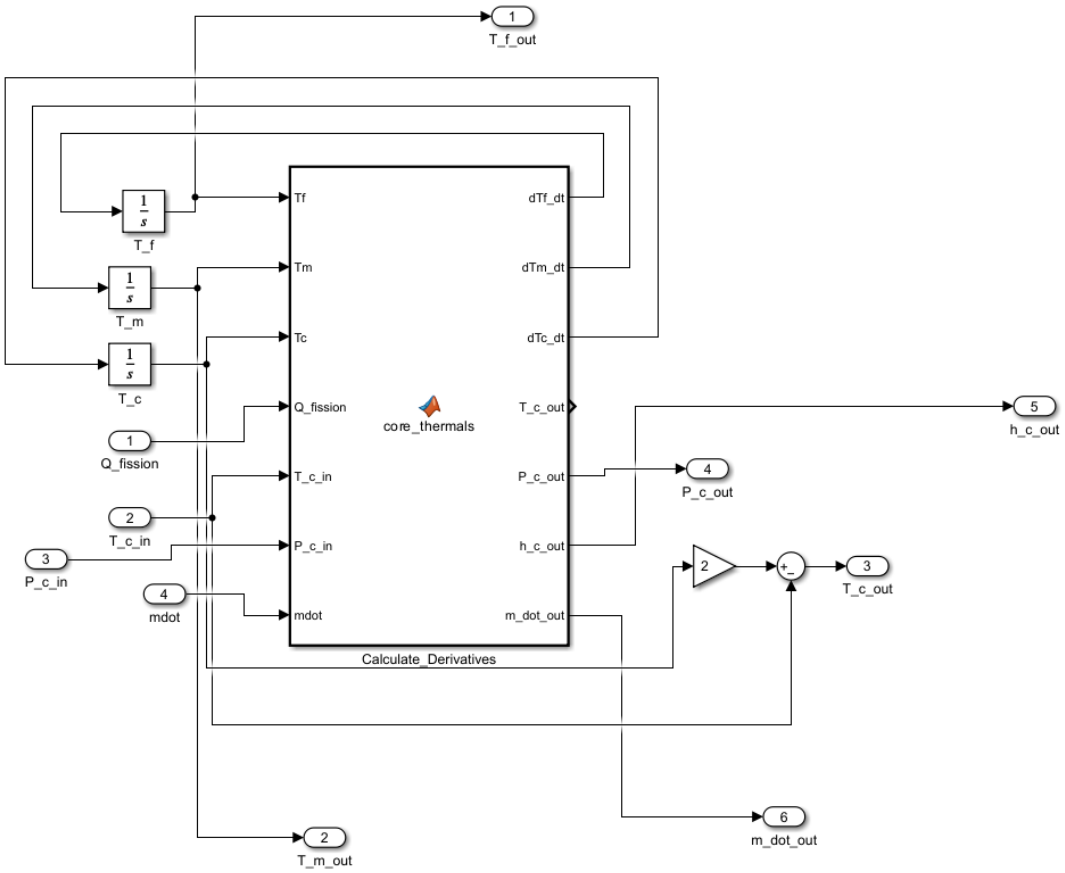


Figure 2: Core Thermals block coded within MATLAB and implemented in Simulink

2.3 Thermal Core Validation & Verification

The Fuel Moderator and Coolant Temperature data were extracted from the Thermal Core Block in our nuclear reactor and is presented in the graph below to verify against existing literature data. The trend shows an excellent correlation with the data from the literature, the main difference being the oscillation, which can be attributed to the lack of controller tuning in the model, which would reduce overshoot and allow for a smoother response. This falls under the development of Process Dynamics and Control which we aim to further implement to stabilise and control our data response in the nuclear reactor model to achieve a more optimised reactor and overall model performance.

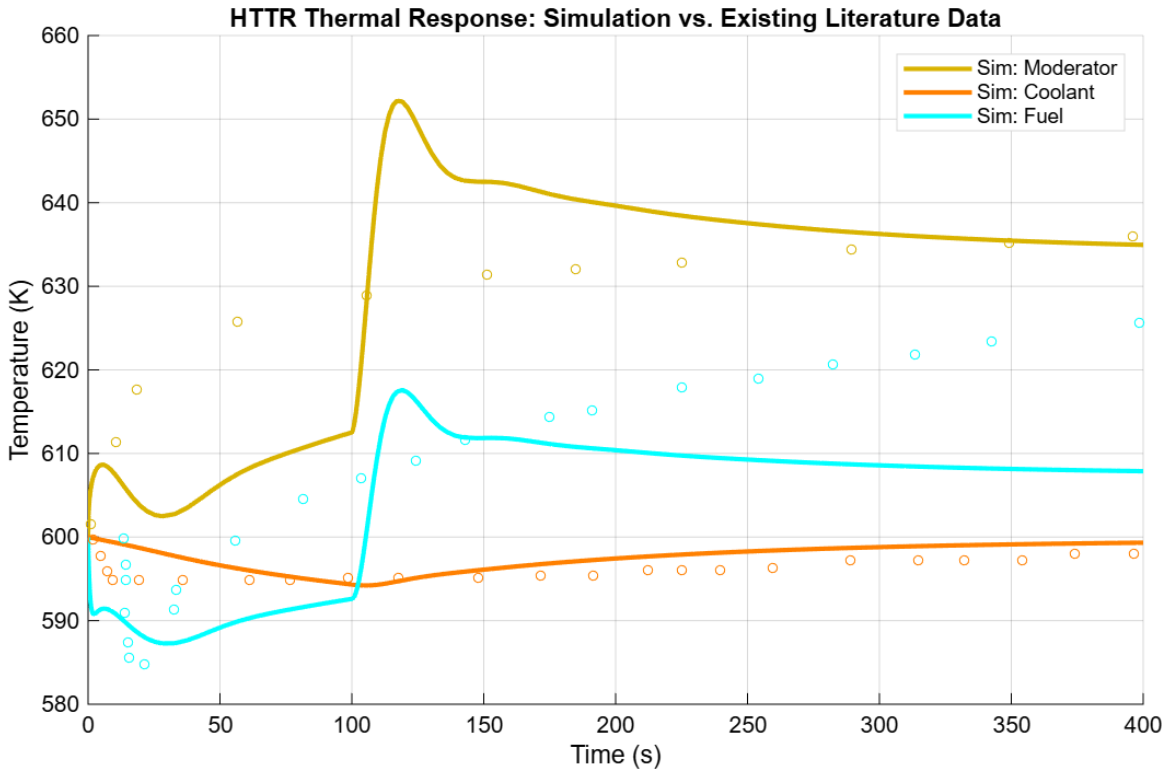


Figure 3: Comparison Data from our Simulink Core Thermals Block with existing literature Data

The simulation results for the High Temperature Engineering Test Reactor (HTTR) demonstrate an excellent correlation with established literature data. In this analysis, the solid lines represent the continuous output from our developed Simulink model, while the discrete circular markers represent the validated literature benchmarks. The model's robustness is evidenced by how closely the solid lines track these markers across the $400\,\mathrm{s}$ transient measurement, specifically capturing the peak Moderator and Fuel temperatures of $652\,\mathrm{K}$ and $618\,\mathrm{K}$ following a reactivity change at $t = 100\,\mathrm{s}$. While the Coolant line shows a very high degree of alignment with the literature dots, the Moderator and Fuel lines exhibit some oscillation and overshoot. This comparative analysis confirms that our model is physically grounded and provides a high-fidelity representation of the reactor's thermal behaviour.

3 Development of Complete Model Complex Configuration

Additional instrumentation was added to model the entire process, including power extraction cycles, heat exchangers, and hybrid desalination unit.

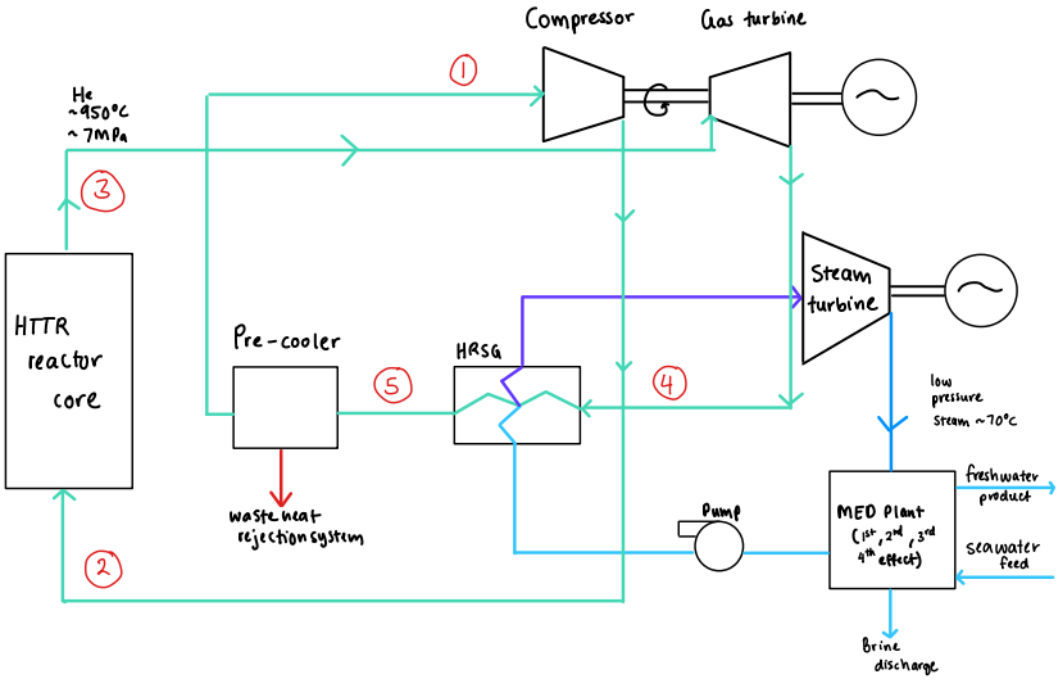


Figure 4: Early sketched out model of complex configuration

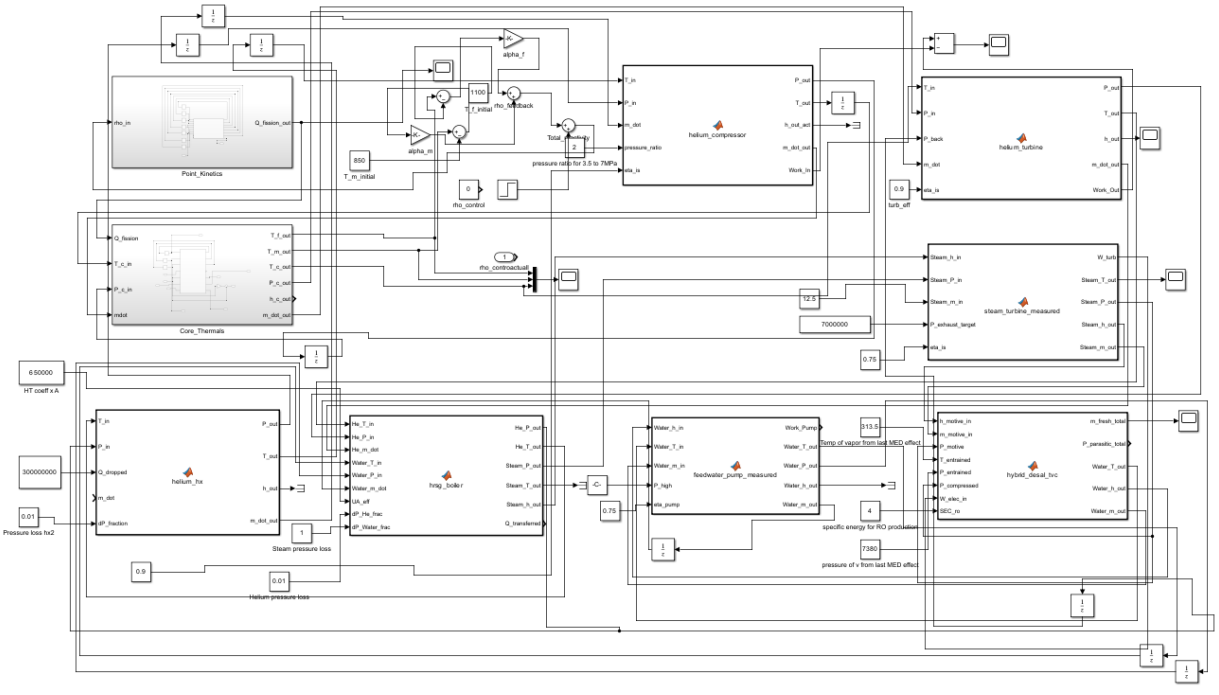


Figure 5: Realised model of complex configuration on Simulink

3.1 Modelling and Analysis Tool for Steady Channel Flow (MATSCF)

MATSCF is a MATLAB-based application created by Imperial College London Student Shapers designed to build upon the fluid mechanic concepts to simulate and visualise steady laminar flow between parallel plates for different conditions, providing insights into how boundary layers and fully developed flow profiles are formed. The governing equations used were:

Continuity equation:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{3.1}$$

Momentum in x-direction:

$$u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) \tag{3.2}$$

Momentum in y-direction:

$$u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial y} + \nu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) \tag{3.3}$$

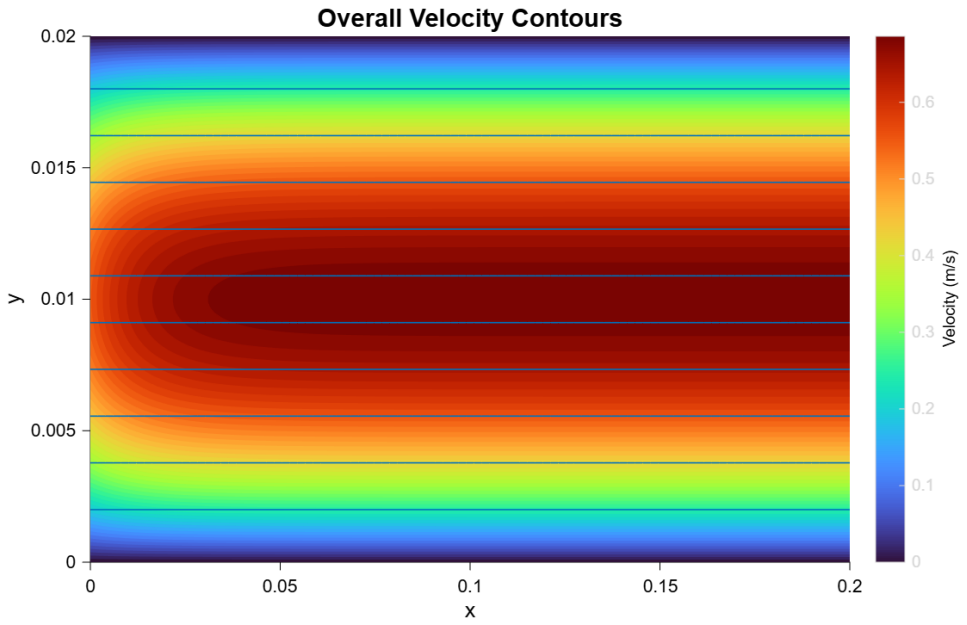


Figure 6: Velocity Contours for Flow Profiles along the Helium Coolant pipes

The velocity contour profile demonstrates the spatial evolution of the fluid flow across the simulated domain. The model exhibits a well-defined parabolic velocity profile characteristic of laminar channel flow, with a peak magnitude of $0.7\,\mathrm{m/s}$ at the centre ($y = 0.01$).

The "no-slip" boundary conditions at the upper and lower walls ($y = 0$) and ($y = 0.02\,\mathrm{m}$) are clearly visualized by the blue low-velocity regions, indicating the development of boundary layers. This spatial data is critical for the Validation and Verification phase of the HTTR model, as it ensures that the heat transfer coefficients used in the Core Thermals block are derived from an accurate representation of the fluid's convective properties. The robustness of the Simulink model is further confirmed by its ability to resolve the transition from the entrance region to a stabilized flow core.

3.2 Thermodynamic Analysis of Power Extraction Cycles

Data measurements from our Simulink model were extracted to construct the thermodynamic temperature-entropy loop cycles below to calculate work output from each turbine loop.

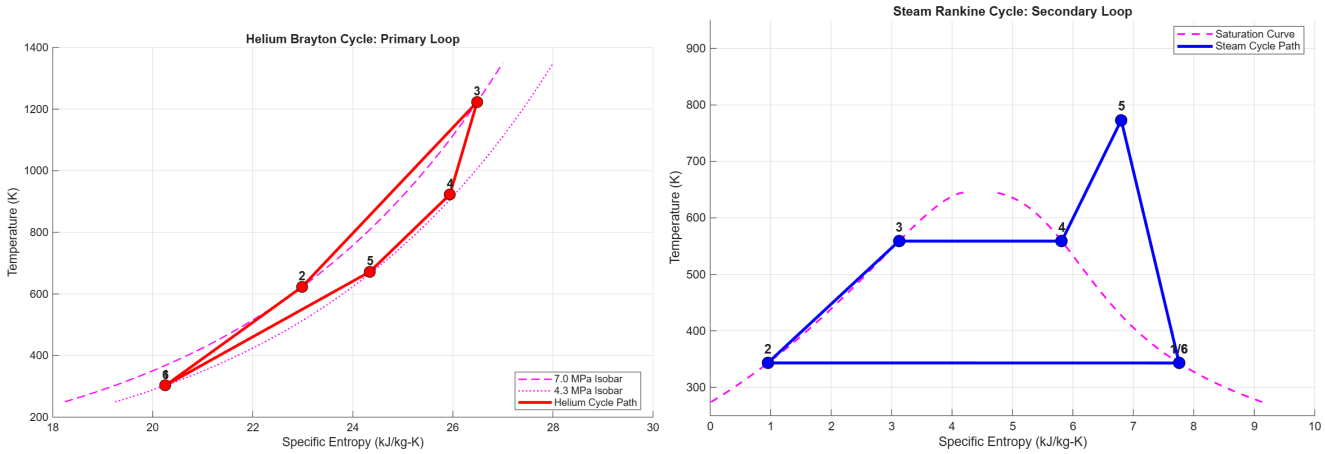


Figure 7: T-S Diagram for Helium Brayton Cycle and Steam Rankine Cycle

Table 4: Comparison between individual and combined cycles

Parameter	Helium Brayton	Steam Rankine	Combined
$T_{\max}$	1220 K	773 K	1220 K
$T_{\min}$	923 K	343 K	343 K
Carnot efficiency	24.5 %	55.6 %	72 %

The combined use of a Helium Brayton and Steam Rankine cycle is preferred because it maximizes energy extraction by utilising a wide temperature gradient. The primary Helium Brayton cycle operates as the high-temperature loop, utilizing the inert properties of helium to extract heat directly from the reactor core at extreme temperatures without the phase-change limitations of water. This is then paired with a Steam Rankine cycle which acts as a bottoming loop, utilizing a steam turbine to capture the remaining exhaust heat from the helium turbine. This secondary loop is essential for desalination projects as it provides the specific steam quality required for thermal distillation processes, MED, while simultaneously generating additional electricity. The equations below were used to calculate the net Work outputs from the power extraction cycles given in the appendix.

$$W_{\text{helium,turb}} = \dot{m}_{\text{He}} \int_{T_{\text{out}}}^{T_{\text{in}}} C_p(T)\, dT = \dot{m}_{\text{He}}(h_{\text{in}} - h_{\text{out}}) \tag{3.4}$$

$$W_{\text{comp}} = \dot{m}_{\text{He}}(h_{\text{out}} - h_{\text{in}}) \tag{3.5}$$

$$\dot{m}_{\text{He}}(h_{\text{He,in}} - h_{\text{He,out}}) = \dot{m}_{\text{water}}(h_{\text{steam,out}} - h_{\text{water,in}}) \tag{3.6}$$

$$W_{\text{steam,turb}} = \dot{m}_{\text{steam}}(h_{\text{in}} - h_{\text{out}})\, \eta_{\text{thermal}} \tag{3.7}$$

where Equation (3.4) gives the helium turbine work output, Equation (3.5) is the compressor work output, Equation (3.6) is the HRSG energy balance, and Equation (3.7) is the steam turbine work output.

3.3 (DEEP) HTTR System Coupling and Hybrid – Desalination Complex

In determining the secondary application of the High Temperature Test Reactor, we decided on utilising a hybrid desalination complex displayed in Figure 8 to utilise waste heat from the Rankine cycle. Our target production is $12{,}000\,\mathrm{m}^3/\mathrm{day}$ of fresh water. DEEP (Desalination Economic Evaluation Program) was used to analyse the effect of nuclear exhaust on hybrid multi-effect distillation. We've decoupled water production from the volatile economics of fossil fuels, providing a stable lifeline for approximately $80{,}000$ people while offsetting $267{,}000$ tonnes of annual carbon dioxide emissions.

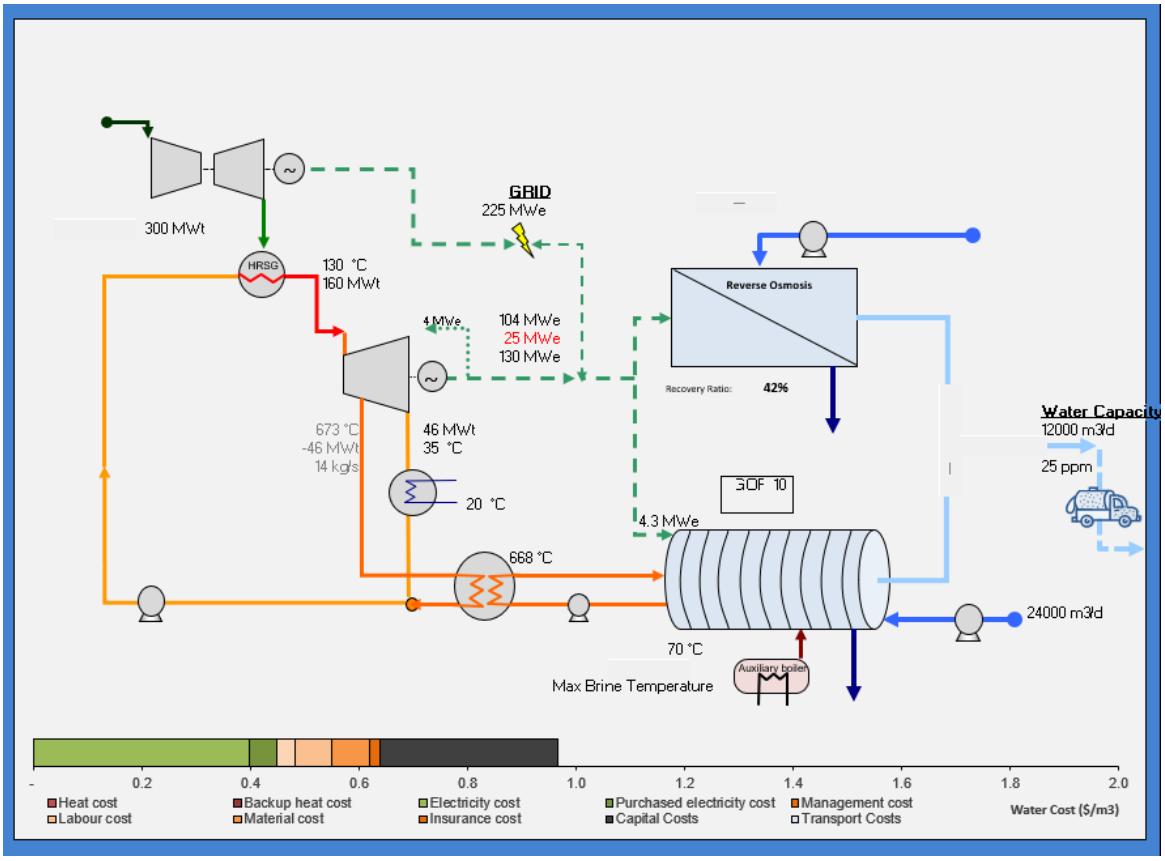
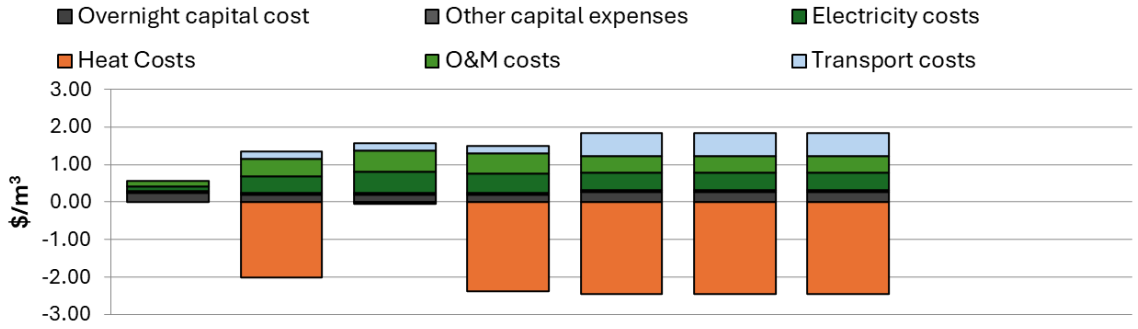


Figure 8: Process Flow Diagram of Complex Configuration in DEEP

From the combined graph analysis in Figure 9, the economic comparison of various desalination configurations coupled with Nuclear Combined Cycles reveals that combined cycle + hybrid desalination configurations are the most financially viable for this model. These setups achieve an optimal balance, featuring Levelized Capital Costs of $\$0.33/\mathrm{m}^3$ and highly competitive Levelized operating costs of $\$\text{-}1.21/\mathrm{m}^3$. This efficiency is bolstered by a consistent Thermal Utilization of $60\,\%$, significantly outperforming the other model alternatives. While Steam cycle + MED offers a deeper operating credit of $\$\text{-}1.50/\mathrm{m}^3$, its lower thermal efficiency and higher capital requirements make it less robust than the hybrid baseline. By maintaining stable power costs of $\$50.8/\mathrm{MWh}$ and balanced lifecycle emissions, the combined cycle + hybrid desalination configurations represent a technically grounded and economically superior choice for integrated nuclear desalination.



Current model run : Outputs Summary			Comb + Hyb	Comb +MED	Gas + MED	Steam +MED	Comb + Hyb	Comb + Hyb	Comb + Hyb
Levelized Capital Costs	0.33	$/m3	0.28	**0.23**	**0.23**	**0.23**	**0.33**	**0.33**	**0.33**
Base plant overnight EPC	0.27	$/m3	0.24	**0.20**	**0.20**	**0.20**	**0.27**	**0.27**	**0.27**
Other	0.05	$/m3	0.04	**0.04**	**0.04**	**0.04**	**0.05**	**0.05**	**0.05**
Levelized operating costs	-1.21	$/m3	0.38	**-1.21**	**0.86**	**-1.50**	**-1.21**	**-1.21**	**-1.21**
Heat	-2.46	$/m3	0.00	**-2.01**	**-0.06**	**-2.38**	**-2.46**	**-2.46**	**-2.46**
Electricity	0.45	$/m3	0.14	**0.46**	**0.57**	**0.53**	**0.45**	**0.45**	**0.45**
O&M	0.19	$/m3	0.24	**0.14**	**0.14**	**0.14**	**0.19**	**0.19**	**0.19**
Transport	0.61	$/m3	0.00	**0.20**	**0.20**	**0.20**	**0.61**	**0.61**	**0.61**
Lifecycle Emissions	47	Mtn/yr	31	31	**15**	**14**	**47**	**47**	**47**
Thermal Utilization	60%		67%	**37%**	**-29%**	**-2%**	**60%**	**60%**	**60%**
Power lost	-25.2	MWe	0	**-62**	0	**-62**	**-25**	**-25**	**-25**
Power used for desalination	4	MWe	4	**13**	**13**	**13**	**4**	**4**	**4**
Power cost	50.8	$/MWh	42.0	42.0	**53.8**	**49.9**	**50.8**	**50.8**	**50.8**

Figure 9: Comparison of various desalination model configurations with Nuclear Combined Cycles

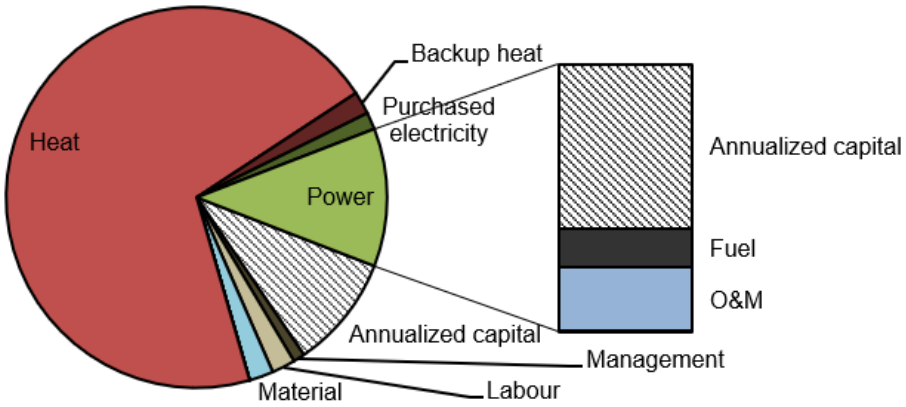


Figure 10: Pie chart of scaled comparison between different capital expenditures

The DEEP economic results provide compelling validation of our project's viability, with the Annualised Capital Cost ($32.2\,\%$) and Heat Cost ($37.1\,\%$) dominating the water price breakdown. In a conventional gas-fired plant, the high heat share represents a precarious vulnerability to global fuel fluctuations; however, our HTGR integration effectively "neutralises" this volatility by reclaiming $673\,^{\circ}\mathrm{C}$ thermal exergy that would otherwise be rejected. This thermodynamic synergy enables Economic Decoupling, as evidenced by the remarkably low Fuel Cost ($3.5\,\%$), ensuring that once the \$1.5B CAPEX is committed, the cost of water for $80{,}000$ residents remains stabilised at \$1.50/m^3 over a 60-year lifecycle. Economically, with our desalination complex we manage to boast a $27\,\%$ IRR to justify a \$1.5 billion Capital Expenditure with a swift 4.3-year payback as displayed in the graph below.

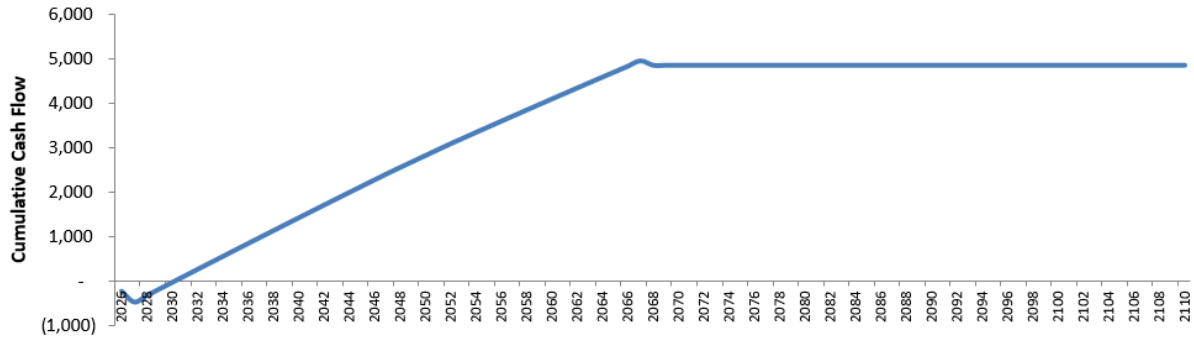


Figure 11: Cumulative Cash Flow of our Model configuration calculated by DEEP

3.4 Performance Metrics of the Multi-Effect Desalination Unit

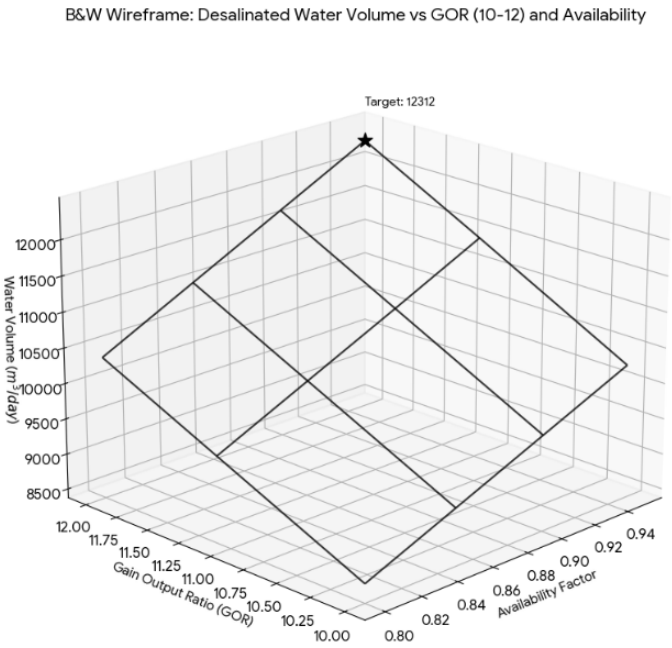


Figure 12: 3D Visualisation of target daily water production based on GOR and Availability Factor

Based on a reactor thermal power of $300\,\mathrm{MW_{th}}$ and a steam mass flow of $12.5\,\mathrm{kg/s}$, the estimated daily water production is calculated as follows:

Total steam input per day:

$$12.5\ \mathrm{kg\,s^{-1}} \times 86{,}400\ \mathrm{s\,day^{-1}} = 1{,}080{,}000\ \mathrm{kg\,day^{-1}}$$

Theoretical water output (at GOR $= 11.1$):

$$1{,}080{,}000\ \mathrm{kg\,day^{-1}} \times 11.1 = 11{,}988{,}000\ \mathrm{kg\,day^{-1}} \approx 12{,}000\ \mathrm{m^3\,day^{-1}}$$

$$12{,}000 \times 0.95 = \mathbf{11{,}400}\ \mathrm{m^3\,day^{-1}} \quad \text{(freshwater produced per day)}$$

4 Sustainability and Net CO_2 Reduction

The energy utilisation factor (EUF) is defined as:

$$\text{EUF} = \frac{\dot{W}_{\text{elec}} + \dot{Q}_{\text{useful}}}{\dot{Q}_{\text{fuel}}} \tag{4.1}$$

Integrating a Helium Brayton primary loop with a Steam Rankine secondary loop creates a high-efficiency combined cycle that maximizes energy utilization. By capturing high-temperature reactor heat via helium and utilizing exhaust to power a bottoming steam cycle, the system achieves a superior EUF over conventional single-loop plants. This configuration is ideal for desalination, providing the precise steam mass flow and temperatures required for high Gain Output Ratios and zero-emission water production. Ultimately, the higher thermal efficiency of this combined approach drives down fuel consumption and significantly lowers carbon dioxide emissions per unit of energy generated, fulfilling the core environmental mandate for modern power plant development.

4.1 Environmental Offset

To quantify the environmental benefits of this project, we must first establish a baseline against conventional fossil-fuel-dependent systems. In water-scarce regions, traditional desalination typically emits $5\ \mathrm{kg\,CO_2\,m^{-3}}$, while regional grid electricity often carries a carbon intensity of $0.74\ \mathrm{t\,CO_2\,MWh^{-1}}$. By displacing these high-carbon sources with nuclear-powered desalination and low-emission electricity, significant greenhouse gas emissions can be avoided. The following section calculates the annual and lifecycle CO_2 savings achieved by transitioning to this cleaner energy alternative over the project's 20–30-year lifespan.

Desalination. Taking a production basis of $12{,}000\ \mathrm{m^3\,day^{-1}}$ at a $95\,\%$ capacity factor, the annual water output and associated displaced CO_2 emissions are:

$$V_{\text{water}} = 12{,}000 \times 0.95 \times 365 = 4{,}161{,}000\ \mathrm{m^3\,yr^{-1}} \tag{4.2}$$

$$E_{\text{desal}} = 4{,}161{,}000 \times 5 = 20{,}805\ \mathrm{t\,CO_2\,yr^{-1}} \tag{4.3}$$

Electricity. Assuming a net electrical output of $40\ \mathrm{MW_e}$ at a $95\,\%$ capacity factor, the annual generation and displaced emissions are:

$$W_{\text{elec}} = 40 \times 0.95 \times 8{,}760 = 332{,}880\ \mathrm{MWh\,yr^{-1}} \tag{4.4}$$

$$E_{\text{elec}} = 332{,}880 \times 0.74 = 246{,}331\ \mathrm{t\,CO_2\,yr^{-1}} \tag{4.5}$$

The total annual CO_2 avoidance is therefore:

$$E_{\text{total}} = E_{\text{desal}} + E_{\text{elec}} = 20{,}805 + 246{,}331 = 267{,}136\ \mathrm{t\,CO_2\,yr^{-1}} \tag{4.6}$$

Over the 30-year project lifespan, this corresponds to approximately 8 million tonnes of CO_2 avoided.

5 Conclusion

This design excels by integrating high fidelity Simulink modelling with a combined Helium Brayton and Steam Rankine cycle, maximising exergy for desalination. Our methodology, validated through MATSCF fluid analysis and Python CoolProp library, ensures a high accuracy comparison against existing literature data and high thermodynamic accuracy. Future work aims to focus on controller performance tuning to further stabilise reactor transients, ensuring an optimised model and sustainable solution for global water scarcity.

Part II: Independent Water Heat Rejection System

1 Introduction

The heat rejection (HR) system is responsible for dissipating thermal energy that cannot be converted to useful work or directed to the desalination complex. For the scaled HTTR configuration developed in Part I, the design thermal rejection duty is $Q_{\mathrm{HR}} = 147.7\,\mathrm{MW_{th}}$ at a helium-side inlet temperature of approximately $399\,^{\circ}\mathrm{C}$ and an ambient air temperature of $20\,^{\circ}\mathrm{C}$.

A key constraint is **water-independence**: the plant is sited in an arid or semi-arid region where consumptive cooling (e.g. wet cooling towers) would undermine the very purpose of the desalination process. The heat rejection system must therefore rely entirely on air or water *in a closed loop* as the ultimate heat sink, while satisfying three competing objectives: low parasitic load, small footprint and absolute safety.

Table 5: Literature review of heat rejection methods

References	Method	Working Fluids	Heat Transfer Coeff.	Parasitic Load	Footprint	Notes
Moore and Hooman (2013)	Air-cooled condenser	Steam $\rightarrow$ Air	$\mathrm{St}\,\mathrm{Pr}^{2/3} \in [0.003, 0.01]$	3–20 %	Large	Well established, widely used in dry regions; performance strongly ambient dependent; low heat transfer coefficient limits compactness.
Zhai & Rubin (2010)	Natural draft dry cooling tower	Water $\rightarrow$ Air	Low U; buoyancy-driven	1–2 %	Very large	Well established, extremely sensitive to local air temperature. Minimises parasitic load yet has impractical footprint.
Kumar, Gireesha and Venkatesh (2026)	Metal porous fins	Humid air	High transient thermal response	<1 %; dependent on ΔP	Compact	Emerging, dependent on many parameters (e.g. Darcy number Da, emissivity ε, porosity ϕ). Repair risks.
Zeyghami, Goswami and Stefanakos (2018)	Radiative cooling panels	Fluid $\rightarrow$ Air	$h_c = 1 + 6V^{0.75}$	<1 %; pump only	Large	Emerging, commonly used in sub-ambient conditions. Heavily wind-dependent.

2 Design Justification

2.1 Candidate Heat Rejection Concepts

Table 6: Weighted Pugh matrix for design concept selection

Criteria	Weight	Air Cond.	Double-Pipe HE	Shell-&-Tube HE	Porous Cube
Parasitic load	0.25	2	4	3	4
Heat transfer coeff.	0.25	3	2	4	5
Footprint	0.20	2	3	2	5
Safety	0.15	4	4	5	3
Maintenance	0.10	3	4	3	2
Cost	0.05	3	4	3	2
Total score	**1.00**	**2.70**	**3.30**	**3.35**	**4.00**

Note: HE = Heat Exchanger; coeff. = coefficient.

As presented in Table 6, the key advantage of utilising porous cube is its immensely high effective surface area A_{eff} at the cost of manufacturability. This allows for excellent heat transfer capabilities (i.e. U), thereby significantly reducing its footprint required to achieve the same heat rejection rate.

2.2 Final Design

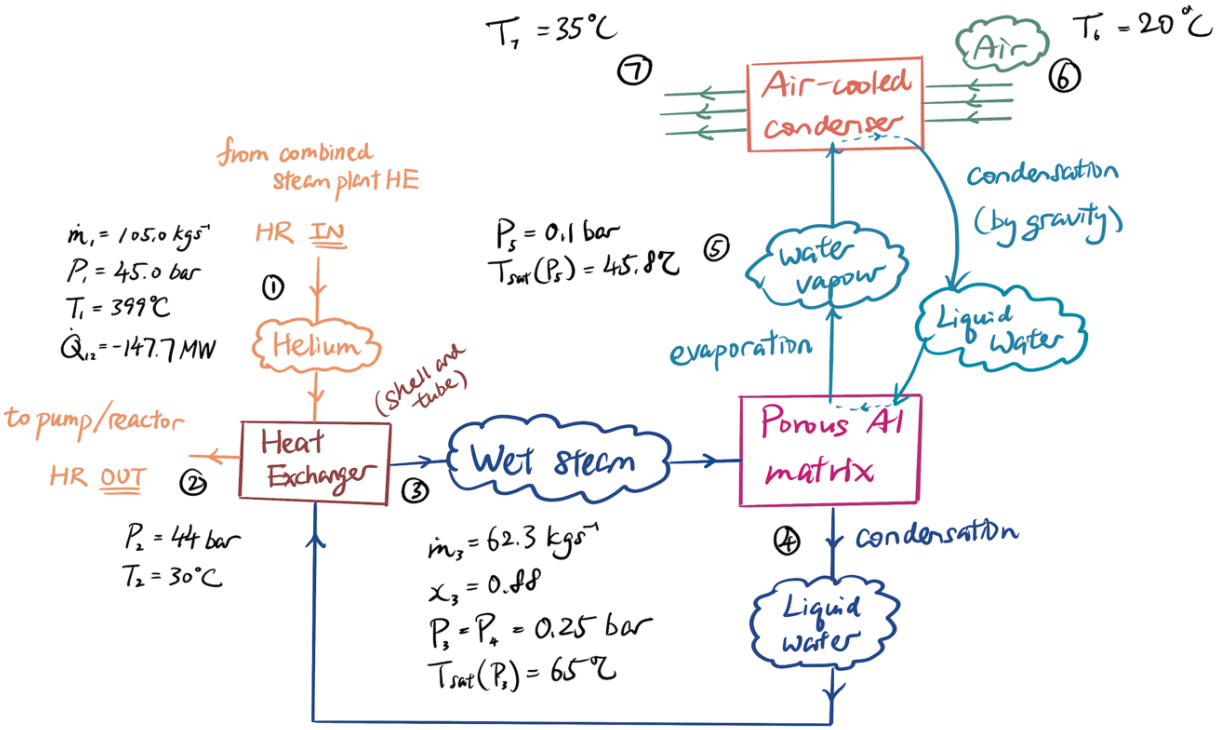


Figure 13: Schematic diagram of final HR design

After experimenting with maximizing the heat transfer for air conduction between pipes or direct pipe to pipe, a neat and small solution shown in the figure below was the porous cube.

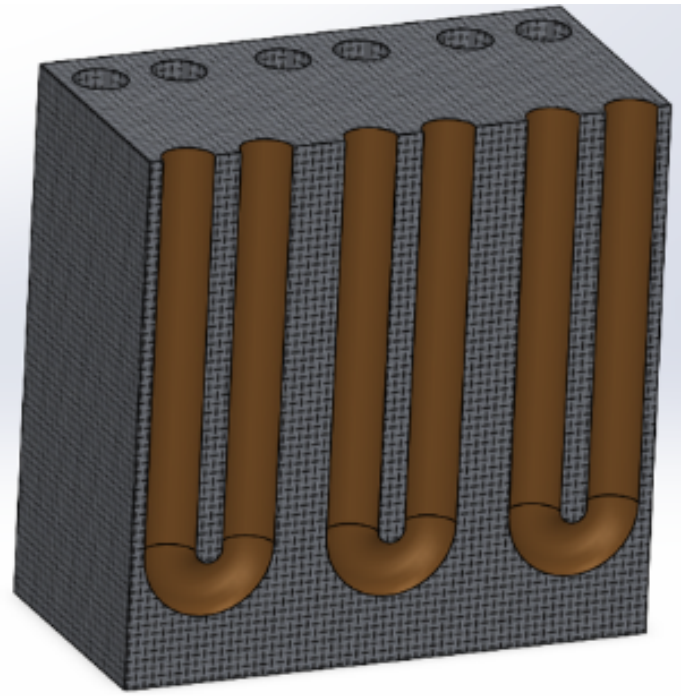


Figure 14: Final design idea for an extra layer from environment and a small footprint

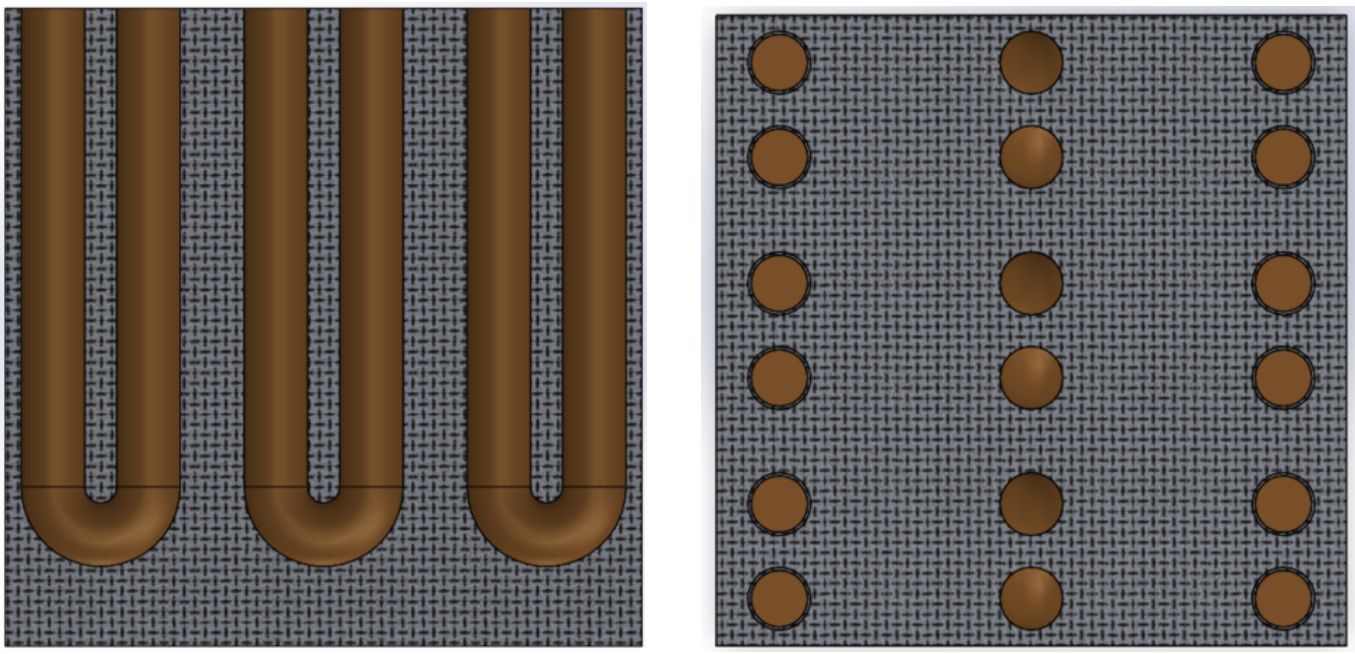


Figure 15: Porous pattern shown in a CAD model with U-shape pipes (left) and top view of pipes rising to air-cooled condenser (right)

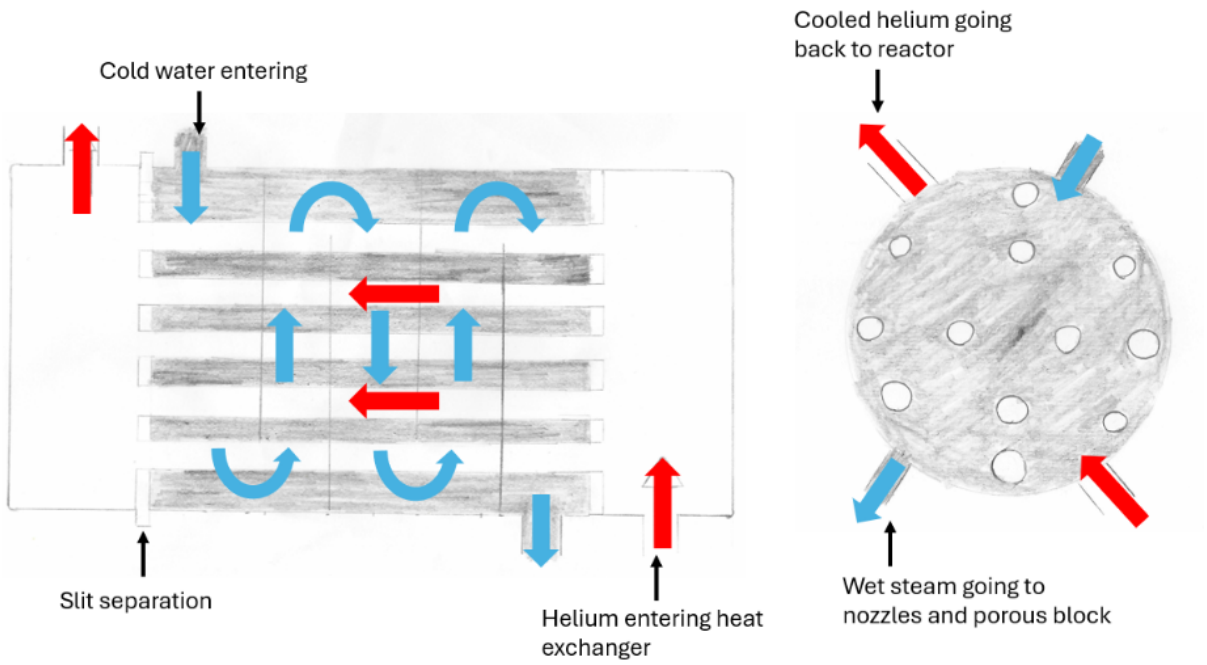


Figure 16: Heat exchanger sketch showing water in grey and pipes for helium in white

2.3 Decision Making and Justification

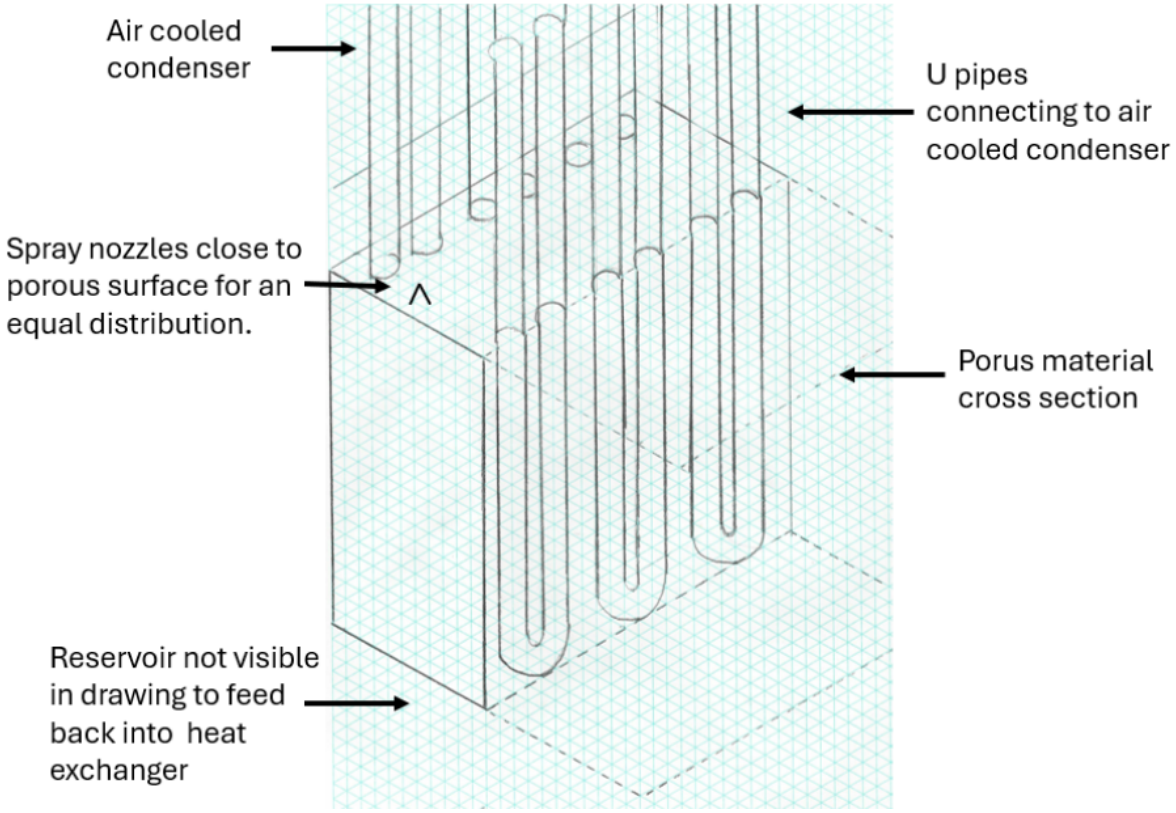


Figure 17: Use of thermosyphon within the porous cube cross section to eliminate need of pumping

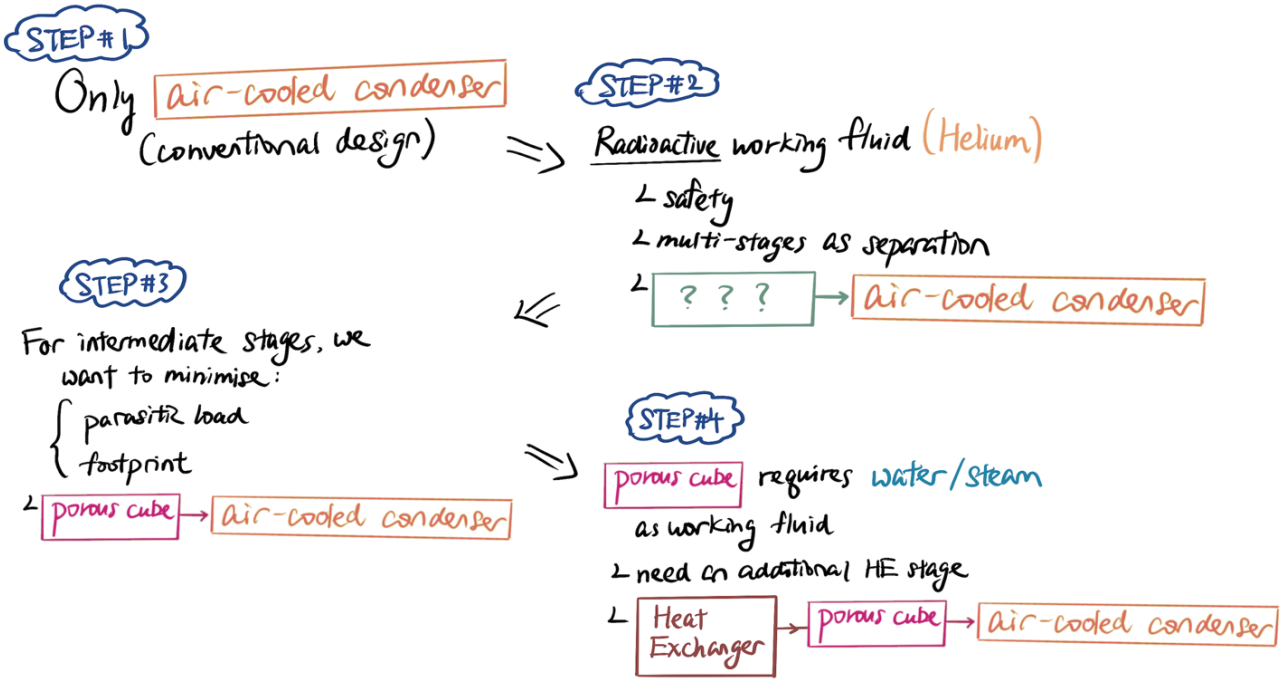


Figure 18: Decision-making process and justification of final design

The final heat rejection design involves three stages of heat exchangers, as presented in Figure 13.

1. Helium-on-water heat rejection, using shell-and-tube heat exchanger.
2. Steam-on-water heat rejection, using porous cube as heat transfer medium.
3. Steam-on-air heat rejection, using air-cooled condenser.

Note the water/steam working fluid loop from porous aluminium to air-cooled condenser utilises a thermosyphon design to eliminate pump power for steam. As illustrated in Figure 18, our final design is multi-staged in consideration for nuclear safety, while minimising parasitic load and footprint.

3 Modeling the System

3.1 Porous foam and thermosyphons

A steady-state sizing model for a combined porous receiver thermosyphon system was developed, performing a sweep on total rejected heat (Q_{total}, a starting value of $147.7\ \text{MW}$, iterated in $\pm 10\ \text{MW}$ steps – this tests system sensitivity). For each Q_{total}, the model calculates:

1. The necessary steam and water mixture mass flowrate from an enthalpy balance, using both inlet vapour quality x_{in} and latent heat.

2. The steam-side condensation heat transfer into a porous copper foam.

This is modelled as Nusselt laminar film condensation on a vertical surface, so we could obtain an order of magnitude steam-side heat transfer coefficient from properties near T_{sat}. The porous foam is modelled as an extended surface, where the available condensation area is estimated from its specific surface area ($A_{\text{foam}} = a_s \cdot V_{\text{foam}}$).

The assumption, and whole principle behind the design, is that the foam improves heat transfer by massively increasing wetted surface area rather than changing the condensation regime. A fin efficiency η_{foam}, based on manufacturer-sourced effective porous conductivity k_{eff} and ligament thickness, does the accounting for conduction losses along the foam structure, giving an effective steam-side resistance in series; tube wall conduction and internal thermosyphon evaporator boiling resistance. Hence, the total heat rate into the thermosyphon is set by R_{tot} and the available temperature driving force.

The thermosyphon count is increased iteratively until per-pipe duty is below the minimum of flooding, critical heat flux, and sonic limits. The evaporator must also have an acceptable ΔT for this count to be outputted. Next, condensate drainage is checked by comparing the available hydrostatic head to the Darcy pressure.

3.2 Parasitic load for Condensate Loop Pump

Volumetric flow:

$$\dot{V} = \frac{\dot{m}}{\rho} = 0.0623\ \text{m}^3\,\text{s}^{-1} \tag{3.1}$$

Diameter calculations:

$$A = \frac{\dot{V}}{v} = 0.02077\ \text{m}^2 \tag{3.2}$$

$$D = \sqrt{\frac{4A}{\pi}} \tag{3.3}$$

Minor loss coefficient (K) for each fitting:

$$\sum K = 6.5 \tag{3.4}$$

Minor (fittings) head:

$$h_m = \frac{(\sum K)\, v^2}{2g} = 2.98\ \text{m} \tag{3.5}$$

Major (pipe friction) head:

$$h_f = f\,\frac{L}{D}\,\frac{v^2}{2g} = 1.32\ \mathrm{m} \tag{3.6}$$

Total head:

$$H = h_f + h_m = 4.3\ \mathrm{m} \tag{3.7}$$

Pump electrical power:

$$P_{\text{pump}} = \frac{\rho g \dot{V} H}{\eta} = 3.51\ \mathrm{kW} \tag{3.8}$$

Fittings:

- $6 \times 90^\circ$ long-radius elbows, $K = 0.45$
- $2\times$ gate valves fully open, $K = 0.15$
- $1\times$ swing check valve, $K = 2.0$

Assumptions:

- Length: $L = 30\ \mathrm{m}$
- Velocity: $v = 3\ \mathrm{m\,s^{-1}}$
- Gravitational acceleration: $g = 9.81\ \mathrm{m\,s^{-2}}$
- Pipe roughness: $\varepsilon = 0.045\ \mathrm{mm}$
- Darcy friction factor: $f = 0.0156$, estimated using a standard explicit friction-factor relation (e.g. Swamee–Jain type forms)
- Pump efficiency: $\eta = 0.75$

3.3 Parasitic load from Dry coolers

Air cooling is based on manufacturer data for 'on the market' units, with a fixed airflow and external area per module. Used in the simulations was the Kelvion RF-NC101E4H model with the following specifications: surface area, $A_{\text{coil}} = 197.1\,\mathrm{m^2}$, airflow, $\dot{V}_{\text{mod}} = 5.39\,\mathrm{m^3/s}$, and dimensions of $2220 \times 1610 \times 1350\,\mathrm{mm}$ (L × W × H).

To determine the number of modules needed, N_{mod}, the air energy capacity per module, Q_{mod} was found, and the effective air energy capacity per module, Q_{eff} was found as follows:

$$Q_{\text{mod}} = \dot{m}_{\text{air}} \times C_p \times \Delta T \tag{3.9}$$

$$Q_{\text{eff}} = \frac{Q_{\text{mod}}}{1.10} \tag{3.10}$$

The number of modules required was found as follows:

$$N_{\text{mod}} = \frac{Q_{\text{total}}}{Q_{\text{eff}}} \tag{3.11}$$

To verify the number of modules needed, a check against the UA requirement was done, with the assumption of crossflow in the dry cooler:

$$\mathrm{LMTD} = \frac{\Delta T_1 - \Delta T_2}{\ln\left(\frac{\Delta T_1}{\Delta T_2}\right)} \tag{3.12}$$

$$UA_{\text{req}} = \frac{Q_{\text{total}}}{0.98 \times \mathrm{LMTD}} = U_{\text{overall}} \times A_{\text{coil}} \times N_{\text{mod}} \tag{3.13}$$

Using the number of modules found, the total airflow $\dot{V}_{\text{total}}$ and estimated fan power P_{fan} were found as follows:

$$\dot{V}_{\text{total}} = \dot{V}_{\text{mod}} \times N_{\text{mod}} \tag{3.14}$$

$$P_{\text{fan}} = \frac{\Delta P_{\text{air}} \times \dot{V}_{\text{total}}}{\eta_{\text{fan}}} \tag{3.15}$$

The thermosyphon count is increased iteratively until per-pipe duty is below the **minimum** of flooding, critical heat flux, and sonic limits. The evaporator must also have an acceptable ΔT for this count to be outputted. The final step is: condensate drainage is checked by comparing the available hydrostatic head to the Darcy pressure.

4 Model Post-Processing

4.1 Simulation Results

Given the baseline performance parameters in Table 7, the output values of the model in Section 3 after running the simulation is shown in Table 8:

Table 7: Baseline performance of the model.

Temperature (°C)	**Heat rejected** (MW_{th})	**Total Parasitic Load** (MW))
20 °C	147.7	3.17

Table 8: Figures of merit of the model from simulation.

Parasitic load per MW_{th} **rejected (kW)**	**Number of modules required**	**Footprint** (m^2)
21.5	1700	60214

4.2 Iterations with HTGR

Below in Figure 20 presents the trend of parasitic loads obtained (either by calculation or simulation) by iteratively exchange parameter values with the HTGR subteam:

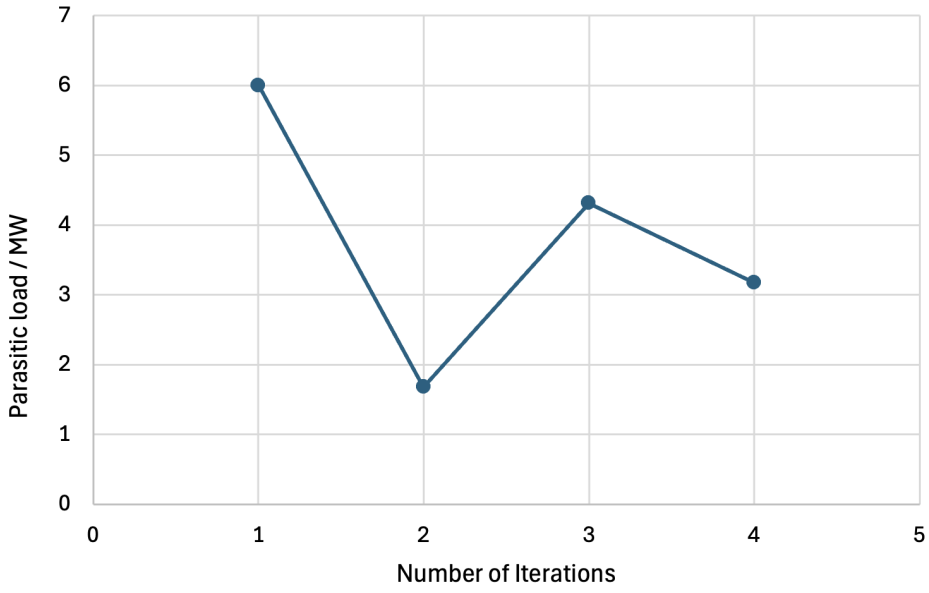


Figure 19: Parasitic load against number of iteration with HTGR subteam.

Iteration #1 took place during literature review, which estimated 3% of ∼200MW heat to be rejected for air-cooled condensers;
Iteration #2 rejected 78MW heat from combined steam plant, resulting in a much smaller value;
Iteration #3 returned to Helium gas heat rejection of 205MW, taking pump power into account;
Iteration #4 utilised the updated 147.7MW value from HTGR after parameter exchange.

4.3 Validation and Verification

A comparison between the parasitic load per MW_{th} rejected from our system against the industry standard was done. Industry trends showed parasitic loads for water independent air cooled heat rejection are typically 4%-6% of power output (Kuzle, Bošnjak and Pandžić, n.d.) due to air's low heat capacity. The predicted 21.5 kW per MW_{th}, ie 2.15% of power output, rejected falls within the same order of magnitude with the industry standard value.

5 Discussion

5.1 Limitations

Ultimately, the main improvement would be to ask for manufacturer performance curves, as thermosyphon internal heat transfer coefficients are represented conservatively rather than calculated. Developing a model based on the dynamics of the inertial pore flow regime and comparing to this one is another suggestion. Even more factors to explore in the future experimentally could be transient thermal storage in the heat receiver, and spatial non-uniformity in the foam (e.g. maldistribution of steam).

5.2 Potential Improvements

A very promising emerging heat rejection technology is radiative cooling panels. Although often used in sub-ambient conditions, it is a passive, material dependent cooling process that requires low maintenance and parasitic power.

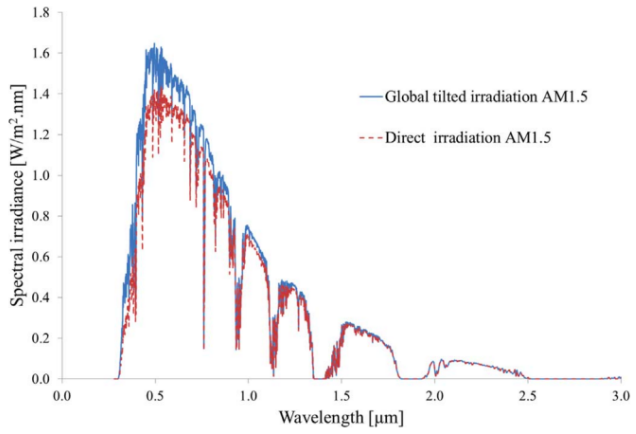


Fig. 2. The global solar spectral irradiance of the AM1.5 spectrum [46].

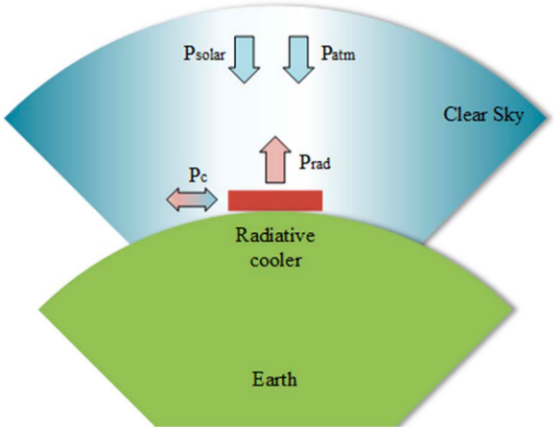


Fig. 3. A schematic representing the heat fluxes for a radiative cooling structure.

Figure 20: (left) AM1.5 global solar spectral irradiance spectrum, (right) schematic representing heat fluxes of a radiative cooling structure. (Zeyghami, Goswami and Stefanakos, 2018) [**?**]

$$q''_{\text{total}} = q''_{\text{rad}}(T_s) + q''_a(T_a) + q''_{\text{solar}} + q''_{\text{conv}} \tag{5.1}$$

Using given steam conditions, the estimated total heat flux is $q''_{\text{total}} \approx 1004.8\,\text{W/m}^2$, with a parasitic load-per-MW_{th} of 0.000208. However, due to its wind dependency, a non-stacking layout would require a large footprint of $53{,}200\,\text{m}^2$ to achieve similar HR targets. (Please refer to Appendix C for details)

6 Conclusion

This design optimised footprint and parasitic load by using porous metal cubes with integrated thermosyphons, enhancing heat transfer per area and reducing pump work in the required safety intermediary loops. A parasitic load of $21.5\ \mathrm{kW}$ per $\mathrm{MW_{th}}$ rejected aligns with industry benchmarks, while reduced loop sizing makes the concept promising for further development.

References

[1] Properties - exxentis - porous aluminium, 05 2021. pages

[2] José Carbia Carril, Álvaro Baaliña Insua, Javier Romero Gómez, and Manuel Romero Gómez. Htr-based power plants' performance analysis applied on conventional combined cycles. *Science and Technology of Nuclear Installations*, 2015:1–11, 2015. pages

[3] Hao Chen, Xiangjun Liu, Jingchong Liu, Fuqiang Wang, and Cunhai Wang. Radiative cooling applications toward enhanced energy efficiency: System designs, achievements, and perspectives. *The Innovation*, page 100999, 06 2025. pages

[4] Jianheng Chen, Lin Lu, and Quan Gong. A new study on passive radiative sky cooling resource maps of china. *Energy Conversion and Management*, 237:114132, 06 2021. pages

[5] Department of Energy. Nuclear 101: What is a high-temperature gas reactor? `https://www.energy.gov/ne/articles/nuclear-101-what-high-temperature-gas-reactor`. Accessed: 1 March 2026. pages

[6] H.T. El-Dessouky and H.M. Ettouney. *Fundamentals of Salt Water Desalination*. Elsevier Science, Amsterdam, 2002. pages

[7] Abdul Samad Farooq, Peng Zhang, Yongfeng Gao, and Raza Gulfam. Emerging radiative materials and prospective applications of radiative sky cooling - a review. *Renewable and Sustainable Energy Reviews*, 144:110910, 07 2021. pages

[8] International Atomic Energy Agency. Nuclear desalination. `https://www.iaea.org/topics/non-electric-applications/nuclear-desalination`. Accessed: 1 March 2026. pages

[9] S. Ito and N. Miura. Studies of radiative cooling systems for storing thermal energy, 08 1989. pages

[10] I. Kuzle, D. Bošnjak, and H. Pandžić. Auxiliary system load schemes in large thermal and nuclear power plants. `https://www.researchgate.net/publication/277248156`, 2015. Accessed: 1 March 2026. pages

[11] Igor Kuzle, Darjan Bošnjak, and Hrvoje Pandži. Auxiliary system load schemes in large thermal and nuclear power plants, 2010. pages

[12] Xing Lu, Peng Xu, Huilong Wang, Tao Yang, and Jin Hou. Cooling potential and applications prospects of passive radiative cooling in buildings: The current state-of-the-art. *Renewable and Sustainable Energy Reviews*, 65:1079–1097, 11 2016. pages

[13] Y. Miyamoto, K. Okada, Y. Inaba, M. Ogawa, S. Shiozawa, I. Minatsuki, K. Oyama, Y. Kiso, and Y. Takenaka. Development of HTGR hydrogen production system in Japan. *Nuclear Engineering and Design*, 233(1–3):363–373, 2004. pages

[14] J Moore, R Grimes, and E J Walsh. Performance analysis of a modular air cooled condenser for a concentrated solar power plant. *Performance Analysis of a Modular Air Cooled Condenser for a Concentrated Solar Power Plant*, pages 715–724, 11 2012. pages

[15] T. Nishihara, K. Hada, and S. Shiozawa. The status of HTTR project and the concept of heat utilization system. In *Transactions of the 4th International Conference on Nuclear Engineering (ICONE-4)*, New Orleans, Louisiana, 1996. Available at: `https://inis.iaea.org/records/dc27b-yc112/preview/37012570.pdf?include_deleted=0` (Accessed: 1 March 2026). pages

[16] M. Ogawa, Y. Inagaki, T. Nishihara, S. Shiozawa, Y. Miyamoto, and A. Fumizawa. Research and development on the HTTR heat utilization system. Technical report, Idaho National Laboratory, Idaho Falls, 1996. Available at: `https://inldigitallibrary.inl.gov/sites/sti/sti/Sort_128404.pdf` (Accessed: 1 March 2026). pages

[17] P.L. Pavan Kumar, B.J. Gireesha, P. Venkatesh, and M.L. Keerthi. Comparative analysis of transient thermal behaviour and efficiency in longitudinal metal porous fin with combined heat and mass transfer under dehumidification conditions. *Applied Thermal Engineering*, 261:125069, 11 2024. pages

[18] Aaswath P. Raman, Marc Abou Anoma, Linxiao Zhu, Eden Rephaeli, and Shanhui Fan. Passive radiative cooling below ambient air temperature under direct sunlight. *Nature*, 515:540–544, 11 2014. pages

[19] M.A. Saeed, H. Al-Ansary, M. Al-Fajad, M. Al-Khawajah, and S.G. Al-Ghamdi. Techno-economic assessment of high-temperature gas-cooled reactors for seawater desalination. *Annals of Nuclear Energy*, 211:110901, 2025. pages

[20] K. Verfondern, T. Nishihara, and M. Ogawa. Safety concept of the HTTR-IS hydrogen production system. *Nuclear Engineering and Design*, 241(12):4641–4649, 2011. pages

[21] E. du Marchie van Voorthuysen and R. Roes. Blue sky cooling for parabolic trough plants. *Energy Procedia*, 49:71–79, 2014. pages

[22] Mehdi Zeyghami, D. Yogi Goswami, and Elias Stefanakos. A review of clear sky radiative cooling developments and applications in renewable power systems and passive building cooling. *Solar Energy Materials and Solar Cells*, 178:115–128, 05 2018. pages

[23] Haibo Zhai and Edward S. Rubin. Performance and cost of wet and dry cooling systems for pulverized coal power plants with and without carbon capture and storage. *Energy Policy*, 38:5653–5660, 10 2010. pages

[24] Tao Zhou, Zhuwang Shao, Zhouquan Sun, Yaogang Li, Qinghong Zhang, Kerui Li, Chengyi Hou, and Hongzhi Wang. High-performance photonic films with enhanced thermal conductivity and high-temperature stability for radiative cooling. *Advanced Functional Materials*, 08 2025. pages

Appendices

Appendix A – Thermodynamic Points

Table 9: Thermodynamic state points for the primary and secondary loops

Loop	Pt.	Component	T (°C)	$\dot{m}$ (kg s^{-1})	P (bar)	Power/Heat (MW)
Primary (He)	1	Compressor inlet	30	105.0	43	-205.1
–	2	Compressor outlet	350	105.0	70	—
–	3	Reactor outlet	950	105.0	70	$+300$
–	4	Gas turbine exit	650	105.0	45	$+147.4$
–	5	HR system inlet	399	105.0	44	-195
–	6	HR system outlet	30	105.0	43	-47.3
Secondary (Water)	A	Feedwater pump out	70	12.5	70	-0.12
–	B	Steam turbine in	500	12.5	70	$+195.0$ (heat)
–	C	Steam turbine out	70	12.5	0.31	$+12.1$ (work)
–	D	Desalination in	70	12.5	0.31	$+182.8$ (heat)

```
function [dTf_dt, dTm_dt, dTc_dt] = core_thermals(Tf, Tm, Tc, Q_fission, T_c_in,
    mdot)
% This function calculates the derivatives for the 3-node HTTR thermal model.

% Thermal Capacities (J/K)
Cf = 5.0e6;   % Fuel thermal capacity
Cm = 1.0e8;   % Moderator (graphite) thermal capacity
Cc = 1.0e6;   % Coolant (helium) thermal capacity

% Heat Transfer Coefficients (W/K)
h_fm = 2.0e5; % Fuel-to-Moderator
h_fc = 4.0e5; % Fuel-to-Coolant
h_mc = 1.5e5; % Moderator-to-Coolant

% Coolant Properties
cp = 5193;    % Specific heat of Helium (J/kg-K)

% Equation 1: Fuel Node
dTf_dt = (Q_fission - h_fm*(Tf - Tm) - h_fc*(Tf - Tc)) / Cf;

% Equation 2: Moderator Node
dTm_dt = (h_fm*(Tf - Tm) - h_mc*(Tm - Tc)) / Cm;

% Equation 3: Coolant Node
dTc_dt = (h_fc*(Tf - Tc) + h_mc*(Tm - Tc) - 2*mdot*cp*(Tc - T_c_in)) / Cc;

end
```

Listing 1: MATLAB function for 3-node HTTR thermal model

```
function [dP_dt, dC1_dt, dC2_dt, dC3_dt, dC4_dt, dC5_dt, dC6_dt, Q_fission] = ...

    Calculate_Kinetics(P, C1, C2, C3, C4, C5, C6, rho_in)

% 1. DEFINE CONSTANTS (Standard U-235 values)

% Delayed Neutron Fractions (beta_i) for 6 groups

beta_i = [0.00025, 0.00138, 0.00122, 0.00266, 0.00164, 0.00035];

% Total delayed neutron fraction

beta_total = sum(beta_i); % This is 0.0075


% Decay Constants (lambda_i, in 1/s)

lambda = [0.0766, 0.2825, 0.6154, 1.634, 5.176, 16.72];


% Prompt Neutron Lifetime (s). For HTTRs (graphite), this is "long".

Lambda = 1.0e-3; % 0.001 seconds


% Nominal Thermal Power (Watts)

% This scales our normalized power (P) to real power (Q)

Q_nominal = 300e6; % 300 MW

% 2. CALCULATE DERIVATIVES (THE ODEs)

% First, calculate the "sum(lambda_i * C_i)" part

sum_lambda_C = lambda(1)*C1 + lambda(2)*C2 + lambda(3)*C3 + ...

               lambda(4)*C4 + lambda(5)*C5 + lambda(6)*C6;


% Equation 1: Power (Neutron Population)

dP_dt = ((rho_in - beta_total) / Lambda) * P + sum_lambda_C;


% Equations 2-7: Precursor Concentrations

dC1_dt = (beta_i(1) / Lambda) * P - lambda(1) * C1;

dC2_dt = (beta_i(2) / Lambda) * P - lambda(2) * C2;

dC3_dt = (beta_i(3) / Lambda) * P - lambda(3) * C3;
```

```
58
59 dC4_dt = (beta_i(4) / Lambda) * P - lambda(4) * C4;
60
61 dC5_dt = (beta_i(5) / Lambda) * P - lambda(5) * C5;
62
63 dC6_dt = (beta_i(6) / Lambda) * P - lambda(6) * C6;
64
65 % 3. CALCULATE THE OUTPUT THERMAL POWER
66
67
68
69 % The block's output is the normalized power (P) scaled by the nominal power.
70
71 Q_fission = P * Q_nominal;
72
73 End
```

Listing 2: Point Kinetics Code

Appendix B – HTGR Code

```
function [P_out, T_out, h_out, m_dot_out, Work_Out] = helium_turbine(T_in, P_in,
    P_back, m_dot, eta_is)

    coder.extrinsic('py.CoolProp.CoolProp.PropsSI');

    P_out = 0.0; T_out = 0.0; h_out = 0.0; m_dot_out = 0.0; Work_Out = 0.0;

    fluid = 'Helium';

    if T_in <= 10 || P_in <= 1000 || m_dot <= 0

        P_out = double(P_in); T_out = double(T_in); m_dot_out = double(m_dot);

    else

        % Initialize variables as doubles before extrinsic calls

        h_in = 0.0;

        s_in = 0.0;

        h_out_s = 0.0;

        T_out_val = 0.0;


        h_in = double(py.CoolProp.CoolProp.PropsSI('H', 'T', double(T_in), 'P',
    double(P_in), fluid));

        s_in = double(py.CoolProp.CoolProp.PropsSI('S', 'T', double(T_in), 'P',
    double(P_in), fluid));


        h_s_val = py.CoolProp.CoolProp.PropsSI('H', 'S', s_in, 'P', double(P_back
    ), fluid);

        h_out_s = double(h_s_val);


        h_out_actual = h_in - (double(eta_is) * (h_in - h_out_s));

        h_out = double(h_out_actual);


        temp_T = py.CoolProp.CoolProp.PropsSI('T', 'H', h_out, 'P', double(P_back
    ), fluid);

        T_out = double(temp_T);


```

```
        P_out = double(P_back);

        m_dot_out = double(m_dot);

        Work_Out = double(m_dot) * (h_in - h_out);

    end

end
```

Listing 3: Helium Turbine Code

```
function [W_turb, Steam_T_out, Steam_P_out, Steam_h_out, Steam_m_out] =
    steam_turbine_measured(Steam_h_in, Steam_P_in, Steam_m_in, P_exhaust_target,
    eta_is)

    % Declare extrinsic functions

    coder.extrinsic('py.CoolProp.CoolProp.PropsSI');



    % 1. Initialize variables to define them as double scalars for the Coder

    fluid = 'Water';

    s_in = 0;

    h_out_s = 0;

    T_out_val = 0;

    Steam_T_out = 0;



    % 2. Inlet Entropy calculation

    % We call the function and immediately cast the output to double

    s_in_mx = py.CoolProp.CoolProp.PropsSI('S', 'H', double(Steam_h_in), 'P',
    double(Steam_P_in), fluid);

    s_in = double(s_in_mx);



    % 3. Actual Expansion logic

    % Get isentropic enthalpy

    h_out_s_mx = py.CoolProp.CoolProp.PropsSI('H', 'S', s_in, 'P', double(
    P_exhaust_target), fluid);

    h_out_s = double(h_out_s_mx);



```

```
    % Now h_out_s is a native double and can be used in this expression:

    h_out_act = double(Steam_h_in) - (double(eta_is) * (double(Steam_h_in) -
    h_out_s));



    % 4. Outputs

    W_turb = double(Steam_m_in) * (double(Steam_h_in) - h_out_act); % Power in
    Watts

    Steam_h_out = h_out_act;

    Steam_P_out = double(P_exhaust_target);



    % Final temperature calculation

    T_out_mx = py.CoolProp.CoolProp.PropsSI('T', 'H', h_out_act, 'P', double(
    P_exhaust_target), fluid);

    Steam_T_out = double(T_out_mx);



    Steam_m_out = double(Steam_m_in);

end
```

Listing 4: Steam Turbine Code

```
function [m_fresh_total, P_parasitic_total, Water_T_out, Water_h_out, Water_m_out
    ] = hybrid_desal_tvc(h_motive_in, m_motive_in, P_motive, T_entrained,
    P_entrained, P_compressed, W_elec_in, SEC_ro)

    % h_motive_in: Enthalpy from LP Turbine outlet (Motive Steam)

    % m_motive_in: Steam flow from Turbine (kg/s)

    % T_entrained/P_entrained: Conditions of vapor from last MED effect



    coder.extrinsic('py.CoolProp.CoolProp.PropsSI');

       Water_h_out = 0; % 1. TVC Entrainment Ratio (Ra)

    % Ra = flow rate of motive steam / flow rate of entrained vapor

    % Using the Power (1994) simplified correlation provided in text

    PCF = 3e-7 * P_motive + 0.95; % Pressure Correction Factor

    TCF = 1.0; % Temp Correction Factor (Simplified for Phase 1)

    Cr = P_compressed / P_entrained; % Compression Ratio
```

```
    % Ra calculation (Semi-empirical model)
    Ra = 0.29 * (Cr^1.19) / (PCF * TCF);
    m_entrained = m_motive_in / Ra;


    % 2. MED Performance (Thermal)
    % Total heating steam = motive + entrained
    m_heating_steam = m_motive_in + m_entrained;
    % GOR is typically higher for TVC (e.g., 10-14)
    GOR_tvc = 11.1;
    m_fresh_med = m_heating_steam * GOR_tvc;


    % 3. RO Process (Electrical)
    m_fresh_ro = W_elec_in / (SEC_ro * 3600); % J/kg conversion


    % 4. Output State for Closed-Loop (To Pump/HRSG)
    % The motive steam and entrained vapor condense into liquid water
    Water_m_out = m_motive_in; % Mass balance of the loop fluid
    Water_T_out = 343.15; % 70 C return temp (saturated liquid)
    Water_h_out = double(py.CoolProp.CoolProp.PropsSI('H', 'T', Water_T_out, 'P',
    P_compressed, 'Water'));


    % 5. System Totals
    m_fresh_total = m_fresh_med + m_fresh_ro;
    P_parasitic_total = W_elec_in; % Load for RO Pumps
end
```

Listing 5: Hybrid Desalination Code

```
unction [He_P_out, He_T_out, Steam_P_out, Steam_T_out, Steam_h_out, Q_transferred
   ] = ...

    hrsg_boiler(He_T_in, He_P_in, He_m_dot, Water_T_in, Water_P_in, Water_m_dot,
```

```
    UA_eff, dP_He_frac, dP_Water_frac)


    % Declare extrinsic for Coder compatibility
    coder.extrinsic('py.CoolProp.CoolProp.PropsSI');


    % Strings for CoolProp
    fluid_he = 'Helium';
    fluid_water = 'Water';
    h_he_in = 0;
    h_he_target = 0;
    h_water_in = 0;
    He_T_out =0;
    Steam_T_out=0;







    % Fluids
    fluid_he    = 'Helium';
    fluid_water = 'Water';


    %% ---------------- 1. INPUT SAFETY ----------------


    % Helium
    T_he_safe = double(He_T_in);
    if T_he_safe < 100
        T_he_safe = 1000;
    end


    P_he_safe = double(He_P_in);
```

```
    if P_he_safe < 1e3
        P_he_safe = 7e6;
    end

    m_dot_he_safe = double(He_m_dot);
    if m_dot_he_safe < 1e-3
        m_dot_he_safe = 85.0;
    end

    % Water
    T_water_safe = double(Water_T_in);
    if T_water_safe < 200
        T_water_safe = 343.15;
    end

    P_water_safe = double(Water_P_in);
    if P_water_safe < 1e3
        P_water_safe = 7e6;
    end

    m_dot_water_safe = double(Water_m_dot);
    if m_dot_water_safe < 1e-3
        m_dot_water_safe = 100.0;
    end

    %% ---------------- 2. CLAMP PRESSURE DROPS ----------------

    dP_He = min(max(double(dP_He_frac),    0.0), 0.9);
```

```
    dP_W  = min(max(double(dP_Water_frac), 0.0), 0.9);


    %% ---------------- 3. THERMODYNAMICS ----------------


    % Helium inlet enthalpy

    h_he_in = double(py.CoolProp.CoolProp.PropsSI( ...

        'H','T',T_he_safe,'P',P_he_safe,fluid_he));


    % Helium target enthalpy (672 K)

    h_he_target = double(py.CoolProp.CoolProp.PropsSI( ...

        'H','T',672.0,'P',P_he_safe,fluid_he));


    % Heat transfer

    Q_transferred = m_dot_he_safe * (h_he_in - h_he_target);


    % Water inlet enthalpy

    h_water_in = double(py.CoolProp.CoolProp.PropsSI( ...

        'H','T',T_water_safe,'P',P_water_safe,fluid_water));


    %% ---------------- 4. OUTLET STATES ----------------


    % Helium outlet

    He_P_out = P_he_safe * (1.0 - dP_He);

    if He_P_out < 1e3

        He_P_out = 1e5;

    end


    h_he_out = h_he_in - Q_transferred / m_dot_he_safe;
```

```
174
175     % Steam outlet
176
177     Steam_P_out = P_water_safe * (1.0 - dP_W);
178
179     if Steam_P_out < 1e3
180
181         Steam_P_out = 1e5;
182
183     end
184
185
186
187     h_steam_out = h_water_in + Q_transferred / m_dot_water_safe;
188
189
190
191     % Enthalpy sanity
192
193     if h_steam_out < 1e5
194
195         h_steam_out = 3e5;
196
197     end
198
199
200
201     Steam_h_out = h_steam_out;
202
203
204
205     %% ---------------- 5. FINAL TEMPERATURE LOOKUPS ----------------
206
207
208
209     He_T_out = double(py.CoolProp.CoolProp.PropsSI( ...
210
211         'T','H',h_he_out,'P',He_P_out,fluid_he));
212
213
214
215     Steam_T_out = double(py.CoolProp.CoolProp.PropsSI( ...
216
217         'T','H',h_steam_out,'P',Steam_P_out,fluid_water));
218
219
220
221 end
```

Listing 6: HRSG Code

Appendix C – Radiative Cooling Panels

Radiative cooling is an emerging technology, that is often used in sub-ambient conditions. There are layers of nuance in radiative panels, considering its heat transfer capabilities depends on several key factors, namely:

$$q''_{total} = q''_{rad}(T_s) + q''_a(T_a) + q''_{solar} + q''_{conv}$$

We will consider the heat flux terms one by one.

C.1 – Atmospheric Radiation q''_{rad}

According to (Martin and Berdahl 1983), atmospheric emissivity may be expressed as a function of dew point temperature as follows:

$$\bar{\epsilon}_a = 0.711 + 0.56\left(\frac{T_{dew}}{100}\right) + 0.73\left(\frac{T_{dew}}{100}\right)^2$$

for $-13°\text{C} < T_{dew} < 24°\text{C}$.

If we refer to weather data at Luton Airport, it is "comfortable" (30%), "humid" (10%), "muggy" (2%) in June to September only, with other times all being "dry".

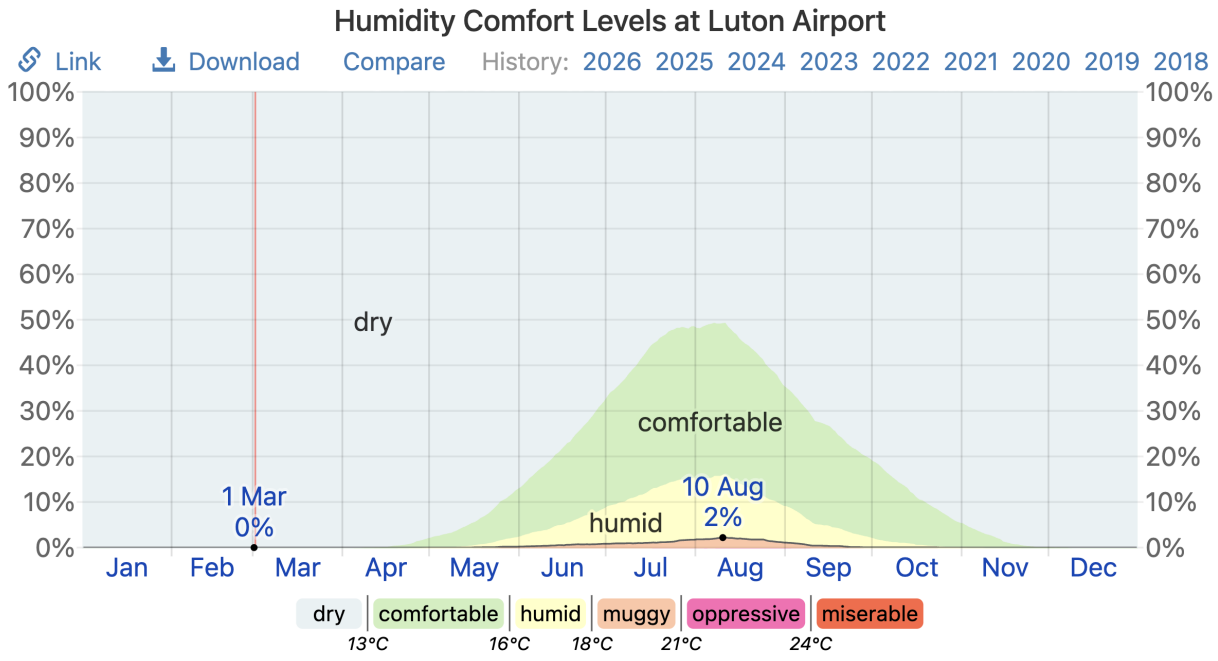


Figure 21: Percentage of time spent at various humidity comfort levels at Luton Airport, categorised by dew point. (Weatherspark.com, 2026)

Thus, we may quantify our $\bar{\epsilon}_a$ and $q''_a = \epsilon_a \sigma T_a^4$, as presented in Table 8 below:

Table 10: Atmospheric heat flux by considering best and worst cases.

Case Type	**Month**	$T_{a,avg}$ / °C	ϵ_a	q''_a / W m^{-2}
Worst case	July	22	0.861	369.72
Bestr case	January	1	0.685	218.92

C.2 – Convection q''_{conv}

Convective heat flux depends heavily on wind speeds. Several relations have been deduced experimentally by Erell and Etzion in 1991 and 2000.

$$h_c = 1 + 6 \times V^{0.75}$$

$$h_c = 1.8 + 3.8 \times V$$

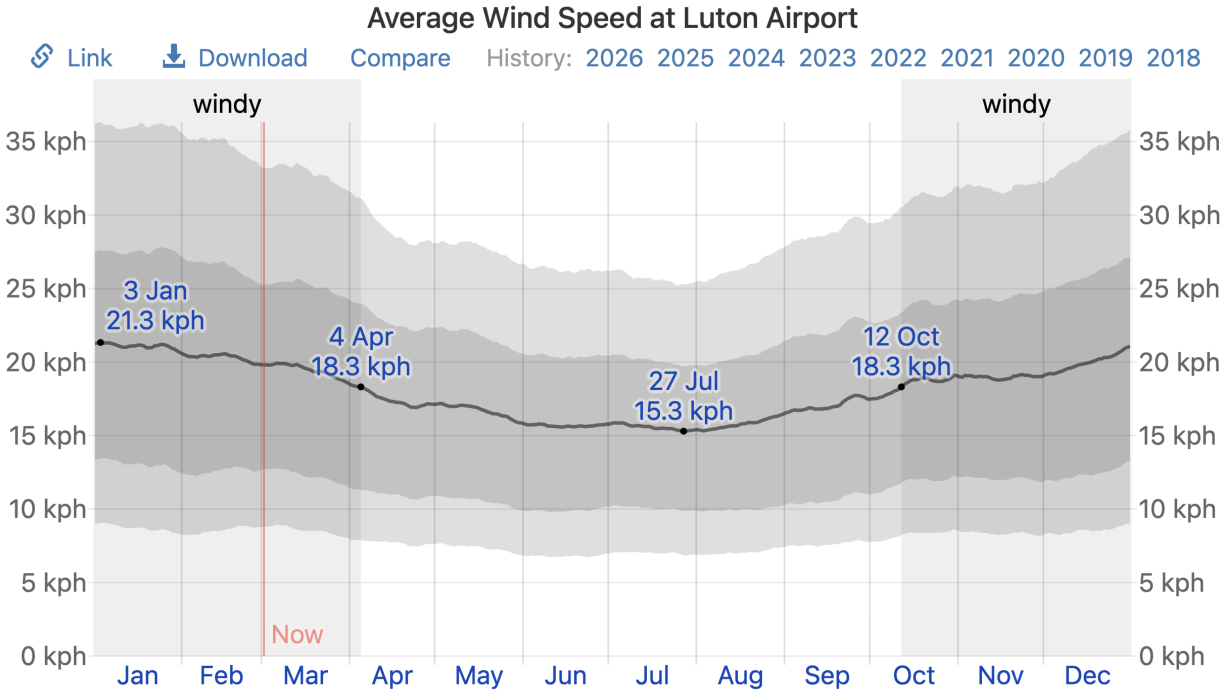


Figure 22: Average of mean hourly wind speeds at Luton Airport. (Weatherspark.com, 2026)

Referring to Figure X, the lowest average wind speed is at July with V = $15.6\,\mathrm{km\,h^{-1}} = 4.33\,\mathrm{m\,s^{-1}}$; whereas highest average wind speed is at January with V = $21.1\,\mathrm{km\,h^{-1}} = 5.86\,\mathrm{m\,s^{-1}}$.

Using $q''_{conv} = h_c(T_s - T_a)$, we get:

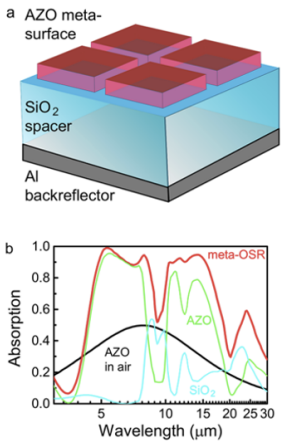
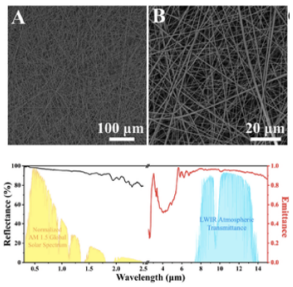
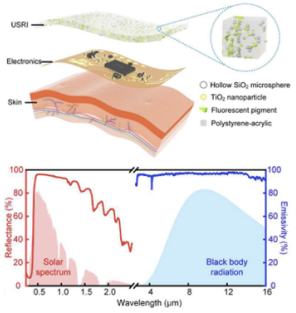
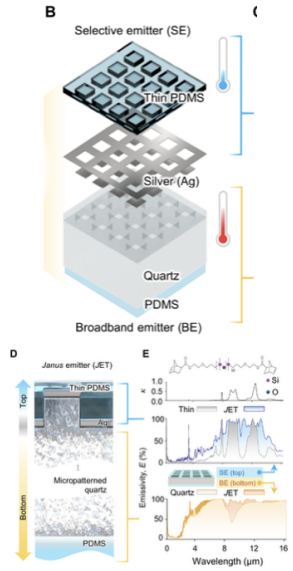
Table 11: Convective heat flux by considering best and worst cases.

Case Type	**Month**	$T_{a,avg}$ / °C	h_c / $\mathrm{W\,m^{-2}\,K^{-1}}$	q''_{conv} / $\mathrm{W\,m^{-2}}$
Worst case	July	22	18.27	774
Bestr case	January	1	24.07	1536

C.3 – Solar Irradiation q''_{conv}

Notably, referring to the AM 1.5 global tilted spectrum, solar irradiance is $964\,\mathrm{W\,m^{-2}}$. However, solar reflectance relies heavily on materials. Liu et al. categorized these radiative panels into three main categories as follows:

Table 12: Three categories of radiative panel materials with examples.

Inorganic-Inorganic	Organic-Organic	Inorganic-organic	
 (Kai Sun et al.)	 (Peng Xu et al.)	 (Jiyu et al.)	 (Se-Yeon Heo et al.)
R_{sol} = 84% $\varepsilon_{broadband}$ = 0.79	R_{sol} = 96% ε_{broad} = 0.95	R_{sol} > 91% ε_{broad} = 0.97	R_{sol} ~ 90% ε_{broad} ~ 0.85
Design strategy: Metal reflection, plasmonic resonances	Design strategy: Sunlight-induced fluorescence, light scattering, molecular vibrations, PhPs resonance	Design strategy: Light scattering, molecular vibrations	Design strategy: Ag layer *isolates* top & bottom in far-IR region
✓ Withstand harsh environments ✗ Spectral performance; Inflexible	✓ Biodegradable; flexible; ease to manufacture ✗ Service life; maintenance	✓ Durable; Adaptable ✓ Advantages of both I-I and O-O	

Se-Yeon Heo et al.'s design appears the most promising given our power plant context. Using the given values, we get a solar irradiation of q''_{solar} = 960 – (1-90%) = $96.4\,\mathrm{W\,m^{-2}}$. Note that q''_{solar} = $0\,\mathrm{W\,m^{-2}}$ at night, therefore it is categorised as the "best case".

C.4 – Emitted IR Radiation by Panels q''_{rad} and Summary

Using the material configuration of Se-Yeon Heo et al., we obtain $q''_{rad} = \epsilon_{broad}\sigma T_s^4 = 629.026\,\mathrm{W\,m^{-2}}$. In summary, we obtain the following results:

Table 13: Summary of best, worst and typical day case of radiative cooling panels.

Heat flux / $\mathrm{W\,m^{-2}}$	**Best case**	**Worst case**	**Typical day case**
q''_{rad}	Unchanged; 629.026	Unchanged; 629.026	Unchanged; 629.026
q''_a	In January; 218.92	In July; 369.72	In warm summer; ∼350
q''_{solar}	At night; 0	During daytime; 96.4	Whole day (avg); 48.2
q''_{conv}	Windy; 1536	No wind; 0	Min. avg wind; 774
q''_{total}	1947	162.9	1004.8

Although radiative panels is a passive cooling technology with its parasitic load-per-MW_{th} being 0.000208, due to its wind dependency, a non-stacking layout would require a large footprint of $53\,200\,\mathrm{m^2}$ to achieve similar HR targets.

Overall, this is a promising technology that could be treated as an add-on to our initial heat rejection design, improving our heat rejection rate while minimising parasitic load. Further investigation is necessary for developing this emerging technology.

Appendix D – HR Code

```
% steam_sponge_thermosyphon_PDC_Q_SWEEP_ALLINONE.m
% ========================================================================
% ONE-FILE: PDC-style step test (steady-state sweep) on Q_total.
% Sweeps down from 147.7 MW in -10 MW steps, then up in +10 MW steps,
% stopping when results become "unrealistic" by simple criteria.
% Outputs a summary table for each Q_total.
% ========================================================================
clear; clc; close all;

%% ---------------- USER SETTINGS ----------------
Q_base_MW    = 147.7;    % baseline [MW]
dQ_MW        = 10;       % step size [MW]
max_steps_each_side = 25; % hard cap so it can't run forever

% "Realism" stop criteria (edit if you want)
max_modules = 5000;         % stop if module count exceeds this
max_parasitic_frac = 0.20; % stop if fan power > 20% of Q_total
min_Q_MW = 10;              % stop if Q gets too small / meaningless

%% ---------------- BUILD Q VECTOR ----------------
Q_list_MW = Q_base_MW;  % start

% sweep down
Qk = Q_base_MW - dQ_MW;
for i=1:max_steps_each_side
    if Qk < min_Q_MW
        break;
    end
    Q_list_MW(end+1) = Qk; %#ok<SAGROW>
    Qk = Qk - dQ_MW;
end

% sweep up
Qk = Q_base_MW + dQ_MW;
for i=1:max_steps_each_side
    Q_list_MW(end+1) = Qk; %#ok<SAGROW>
    Qk = Qk + dQ_MW;
end

% Put baseline first, then decreasing, then increasing (as you described)
Q_down = sort(Q_list_MW(Q_list_MW<=Q_base_MW),'descend');  %
   147.7,137.7,127.7,...
Q_up   = sort(Q_list_MW(Q_list_MW>=Q_base_MW),'ascend');   % 147.7,157.7,...
Q_vec_MW = [Q_down, Q_up(2:end)];

%% ---------------- RUN SWEEP ----------------
n = numel(Q_vec_MW);

% Preallocate containers (use NaN, then fill)
Q_MW        = nan(n,1);
N_modules   = nan(n,1);
Pfan_MW     = nan(n,1);
Vdot_m3s    = nan(n,1);
UA_margin   = nan(n,1);
Tair_out_C  = nan(n,1);
```

```
N_TPCT      = nan(n,1);

stop_idx = n;  % will shorten if stop criteria hit

for k=1:n
    Q_MW(k) = Q_vec_MW(k);
    out = steam_sponge_model_run(Q_MW(k)*1e6);

    % Pull outputs (guarded)
    if isfield(out,'N_modules'); N_modules(k) = out.N_modules; end
    if isfield(out,'P_fan_total_W'); Pfan_MW(k) = out.P_fan_total_W/1e6; end
    if isfield(out,'Vdot_air_total_m3s'); Vdot_m3s(k) = out.Vdot_air_total_m3s;
   end
    if isfield(out,'UA_margin'); UA_margin(k) = out.UA_margin; end
    if isfield(out,'T_air_out_C'); Tair_out_C(k) = out.T_air_out_C; end
    if isfield(out,'N_TPCT_total'); N_TPCT(k) = out.N_TPCT_total; end

    % ---- stop criteria ----
    parasitic_frac = Pfan_MW(k)*1e6 / (Q_MW(k)*1e6); % W/W
    if (~isnan(N_modules(k)) && N_modules(k) > max_modules) || ...
       (~isnan(parasitic_frac) && parasitic_frac > max_parasitic_frac)
        stop_idx = k;
        break;
    end
end

% Trim vectors if stopped early
Q_MW       = Q_MW(1:stop_idx);
N_modules  = N_modules(1:stop_idx);
Pfan_MW    = Pfan_MW(1:stop_idx);
Vdot_m3s   = Vdot_m3s(1:stop_idx);
UA_margin  = UA_margin(1:stop_idx);
Tair_out_C = Tair_out_C(1:stop_idx);
N_TPCT     = N_TPCT(1:stop_idx);

% Baseline index (should be first element)
idx0 = find(abs(Q_MW - Q_base_MW) < 1e-6, 1, 'first');

% Percent deltas vs baseline
dPfan_pct    = 100*(Pfan_MW - Pfan_MW(idx0)) / Pfan_MW(idx0);
dModules_pct = 100*(N_modules - N_modules(idx0)) / N_modules(idx0);

%% ---------------- SUMMARY TABLE ----------------
T = table(Q_MW, N_modules, N_TPCT, Pfan_MW, Vdot_m3s, Tair_out_C, UA_margin,
   dPfan_pct, dModules_pct, ...
    'VariableNames', {'Q_total_MW','N_modules','N_TPCT_total','P_fan_MW','
   Vdot_air_m3s','T_air_out_C','UA_margin','dPfan_pct','dModules_pct'});

disp('==================== RESULTS SUMMARY (each step = new steady state)
   ====================');
disp(T);

%% ---------------- PLOTS ----------------
figure; plot(Q_MW, Pfan_MW, 'o-','LineWidth',1.5); grid on;
xlabel('Q_{total} [MW]'); ylabel('Fan parasitic power [MW]');
title('Steady-state sensitivity: P_{fan} vs Q_{total}');

```

```
figure; plot(Q_MW, dPfan_pct, 'o-','LineWidth',1.5); grid on;
xlabel('Q_{total} [MW]'); ylabel('\Delta P_{fan} vs baseline [%]');
title('Reactivity metric: % change in parasitic load');

%% ========================================================================
% MODEL WRAPPER (your vendorized sizing model, converted to a function)
% ========================================================================
function out = steam_sponge_model_run(Q_total_in)
% 0) SYSTEM-LEVEL INPUTS (TEAM VALUES)
% ========================================================================
% These are the top-level numbers your team already agreed on.
Q_total = Q_total_in;           % [W] swept input

% Ambient air side (TEAM SLIDES)
T_air_in_C  = 20;                % [ C ] ambient air inlet temperature (TEAM SLIDES)
P_amb       = 101325;            % [Pa] ambient pressure (TEAM SLIDES)
dT_air      = 15;                % [K] allowed air temperature rise (TEAM SLIDES)
cp_air      = 1005;              % [J/kg-K] air specific heat (TEAM SLIDES)
rho_air     = 1.177;             % [kg/m^3] air density at design point (TEAM SLIDES)
dP_air      = 225;               % [Pa] allowed air pressure drop (TEAM SLIDES)
eta_fan     = 0.65;              % [-] fan+motor+drive efficiency (TEAM SLIDES)
crosswind_multiplier = 1.10;     % [-] airflow derate / multiplier (TEAM SLIDES)

% Coil "lumped" performance (TEAM SLIDES)
% This U_overall is a *design-level* overall coefficient (air-side dominated).
% In even more detailed coil design one would compute it from fin efficiency + j/f etc.
U_overall = 40;                  % [W/m^2-K] overall U (TEAM SLIDES)
F_LMTD    = 0.98;                % [-] LMTD correction factor (TEAM SLIDES)

% Steam condition (TEAM SLIDES)
Tsat_C   = 65;                   % [ C ] steam saturation temperature in receiver (TEAM SLIDES)
Psat_bar = 0.25;                 % [bar] saturation pressure (TEAM SLIDES)
x_in     = 0.88;                 % [-] steam quality at inlet (TEAM SLIDES)
x_out    = 0.0;                  % [-] quality after full condensation (design target)
h_fg     = 2.346e6;              % [J/kg] latent heat at ~65 C  (TEAM SLIDES)

% Gravity (physical constant)
g = 9.81;                        % [m/s^2] gravity

% Condenser surface temperature target
% Must exceed air outlet temperature for heat to flow to air.
T_cond_surface_C = 62;           % [ C ] (TEAM / DESIGN TARGET)

%% ========================================================================
% 1) FLUID PROPERTIES (WATER/STEAM + AIR)
% ========================================================================
% In a perfect model one would use IAPWS-IF97 or CoolProp.
% Here, we use a simple saturation-table helper for the 50   100C    range.
% This is adequate for preliminary sizing near 65 C .

st = satWater_simple(Tsat_C);    % [sat properties] (internal function below)
```

```
159
160 % Enthalpy of wet steam mixture in and saturated liquid out:
161 %   h_in = hf + x*hfg,  h_out = hf (since x_out=0).
162 % We use h_fg from your slides for consistency with your work, but the sat
163 % table provides a similar value.
164 h_in  = st.hf + x_in*h_fg;        % [J/kg] (TEAM h_fg + sat hf)
165 h_out = st.hf + x_out*h_fg;       % [J/kg]
166 dh    = h_in - h_out;             % [J/kg] heat removed per kg mixture
167
168 % Mass flow required to reject Q_total:
169 m_dot_mix   = Q_total/dh;         % [kg/s] total mixture mass flow (CALC)
170 m_dot_vapor = x_in*m_dot_mix;     % [kg/s] incoming vapor that condenses (CALC)
171
172 % Air dynamic viscosity and thermal conductivity:
173 % Your slides  d o n t  specify mu_air and k_air. For preliminary work, we set
174 % typical values near ~25   30C   .
175 % If you want higher fidelity, replace these with CoolProp calls.
176 mu_air  = 1.90e-5;                % [Pa s] typical air viscosity near room temp
    (ASSUMPTION)
177 k_air   = 0.027;                  % [W/m-K] typical air conductivity near room
    temp (ASSUMPTION)
178 Pr_air  = cp_air*mu_air/k_air;    % [-] Prandtl number (CALC)
179
180 %% ========================================================================
181 % 2) THERMOSYPHON (TPCT) GEOMETRY + MATERIAL (VENDOR-ORIENTED)
182 % ========================================================================
183 % Here we choose a geometry that matches *published* vendor language:
184 % ACT brochure for split-loop thermosyphon coil: "Half-inch diameter tubes"
185 % (interpreted as ~12.7 mm OD). (ACT HVAC brochure PDF, see link above.)
186
187 % --- Tube outside diameter (OD) ---
188 tp.OD = 0.0127;                   % [m] 1/2 inch = 12.7 mm (ACT brochure gives
    1/2-inch tubes)
189
190 % --- Tube wall thickness and inside diameter (ID) ---
191 % ACT brochure does not provide wall thickness; for a defendable starting
192 % point we use a standard copper tube "Type L" wall thickness of ~0.040"
193 % for nominal 1/2" size (example supplier listing).
194 % Source (example): https://www.coppertubingsales.com/collections/copper-tubing-
    type-l
195 tp.t_wall = 0.001016;             % [m] 0.040 inch = 1.016 mm (ASTM B88 Type L
    typical)
196 tp.ID = tp.OD - 2*tp.t_wall;      % [m] derived internal diameter (CALC)
197
198 % --- Lengths ---
199 % These MUST come from the mechanical layout (CAD) and system constraints.
200 % Earlier code used Lev=1.5 m and Lco=3.0 m. We keep those as the
201 % current "team layout assumption" until a vendor drawing / final CAD exists.
202 tp.Lev = 1.50;                    % [m] evaporator length inside sponge (TEAM CAD
     ASSUMPTION)
203 tp.Lco = 3.00;                    % [m] condenser length in air coil region (TEAM
     CAD ASSUMPTION)
204
205 % --- Tube wall thermal conductivity ---
206 % If these TPCTs are copper tubes, copper k ~ 401 W/m-K at ~0 C  (order is
207 % similar at room temp). We use 390 W/m-K as a conservative near-room value.
208 % Source: EngineeringToolbox copper k table.
```

```
% https://www.engineeringtoolbox.com/thermal-conductivity-metals-d_858.html
tp.k_wall = 390;                    % [W/m-K] copper tube wall conductivity (
    REFERENCE TABLE)

% --- Areas for heat transfer and wall conduction resistances ---
Aev_od = pi*tp.OD*tp.Lev;           % [m^2] external evaporator area (tube outside)
     (CALC)
Aev_id = pi*tp.ID*tp.Lev;           % [m^2] internal boiling area (CALC)
Aco_id = pi*tp.ID*tp.Lco;           % [m^2] internal condensation area (CALC)

% Conduction resistance of tube wall (cylindrical log conduction):
R_wall_ev = log(tp.OD/tp.ID)/(2*pi*tp.k_wall*tp.Lev);   % [K/W] (CALC)
R_wall_co = log(tp.OD/tp.ID)/(2*pi*tp.k_wall*tp.Lco);   % [K/W] (CALC)

%% =======================================================================
% 3) POROUS RECEIVER ("SPONGE/FOAM") (RECEMAT-BASED PLACEHOLDERS)
% =======================================================================
% For the foam, we select a Recemat grade whose datasheet explicitly gives
% porosity and specific surface area density.
%
% Example chosen grade: Recemat copper foam "Cu-1116"
% Datasheet gives:
%   Porosity = 95%   (eps = 0.95)
%   Specific surface     1000 m^2/m^3 (a_s = 1000)
% Source:
%   https://www.recemat.nl/wp-content/uploads/2020/08/Datasheet_Cu.pdf

foam.eps = 0.95;                    % [-] porosity (Recemat datasheet)
foam.a_s = 1000;                    % [m^2/m^3] specific surface area density (
    Recemat datasheet)

% The following are NOT typically given as a single number on Recemats short
% datasheet and should be requested from the vendor (or measured):
%   - effective thermal conductivity k_eff at your temperature and boundary
%   - permeability K_perm for liquid drainage (or pressure-drop curve)
%   - ligament thickness t_lig (or micrograph / strut thickness)
%
% Until you have those, we keep the same values you had in your updated code,
% but mark them clearly as "NEEDS VENDOR DATA".
foam.k_eff  = 5.8;                  % [W/m-K] NEED VENDOR DATA (temporary team
    estimate)
foam.K_perm = 2e-8;                 % [m^2]   NEED VENDOR DATA (temporary
    assumption)
foam.t_lig  = 0.0008;               % [m]     NEED VENDOR DATA (temporary
    assumption)

% Foam thickness between steam surface and TPCT tube (geometric design)
% This should come from your CAD.
foam.t_receiver = 0.020;            % [m] (TEAM CAD ASSUMPTION)

% Optional enhancement factor for condensation on porous surfaces.
% Use 1.0 unless you have test/literature support for your exact foam.
foam.F_cond_enh = 1.0;              % [-] NEED TEST/LITERATURE (default 1.0)

% Geometry: treat foam as annulus around tube evaporator section
r0 = tp.OD/2;                       % [m] tube outer radius (CALC)
r1 = r0 + foam.t_receiver;          % [m] outer foam radius (CALC)
```

```
260
261 A_cs_foam   = pi*(r1^2 - r0^2);   % [m^2] foam flow cross-section per TPCT (CALC)
262 V_foam      = A_cs_foam * tp.Lev; % [m^3] foam volume per TPCT evaporator (CALC)
263 A_foam_geom = foam.a_s * V_foam;  % [m^2] geometric internal foam area (CALC)
264
265 %% ========================================================================
266 % 4) STEAM-SIDE CONDENSATION HTC (NUSSELT FILM CONDENSATION)
267 % ========================================================================
268 % Physical meaning:
269 %   - Steam at Tsat condenses on cooler surfaces.
270 %   - A liquid film forms; heat flows through the film by conduction.
271 % Nusselts laminar-film condensation theory gives a baseline average h.
272 %
273 % We use a standard vertical-surface average form:
274 %   h = 0.943 * [ rhoL*(rhoL-rhoV)*g*hfg*kL^3 / (muL*L* T ) ]^(1/4)
275 %
276 % This gives order-of-magnitude realistic condensation HTCs without guessing.
277 % (You can later upgrade this to forced convection / shear-driven models if
278 % steam velocities are high.)
279
280 w = satWater_simple(Tsat_C);      % sat densities and hfg from table (internal
    function)
281
282 mu_l = muWater_simple(Tsat_C);    % [Pa s] liquid viscosity (simple correlation)
283 k_l  = kWater_simple(Tsat_C);     % [W/m-K] liquid conductivity (simple
    correlation)
284
285 % Key sizing assumption: the temperature drop driving condensation.
286 % This depends on how cool the receiver surface is relative to Tsat.
287 dT_steam_receiver = 8;            % [K] TEAM DESIGN ASSUMPTION (explicit)
288
289 h_nusselt = hFilmCond_NusseltVertical( ...
290     w.rho_l, w.rho_v, mu_l, k_l, h_fg, g, tp.Lev, dT_steam_receiver);  % (
    CORRELATION)
291
292 h_steam = foam.F_cond_enh * h_nusselt; % apply optional enhancement (default 1.0)
293
294 % Foam extended-surface efficiency:
295 % We treat ligaments as thin fins of characteristic thickness t_lig and
296 % length ~ foam thickness. This is conservative but physically motivated.
297 L_fin = foam.t_receiver;          % [m] fin length scale (geometry)
298 m = sqrt(2*h_steam/(foam.k_eff*foam.t_lig));  % [1/m] fin parameter (CALC)
299 eta_foam = tanh(m*L_fin)/(m*L_fin);           % [-] fin efficiency (CALC)
300 eta_foam = max(min(eta_foam,1),0.05);         % clamp to sane range (CODE SAFETY)
301
302 % Effective steam-side area per TPCT:
303 %   A_eff = tube OD area + (foam internal area * fin efficiency)
304 A_eff_steam = Aev_od + eta_foam*A_foam_geom;  % [m^2] (CALC)
305
306 % Steam-side thermal resistance (condensation film):
307 R_steam = 1/(h_steam*A_eff_steam);            % [K/W] (CALC)
308
309 %% ========================================================================
310 % 5) INTERNAL EVAPORATOR BOILING HTC (INSIDE TPCT EVAPORATOR)
311 % ========================================================================
312 % In a wickless thermosyphon evaporator, heat transfer can resemble pool
313 % boiling / thin-film evaporation on the inner wall.
```

```
%
% In preliminary design one typically does *not* compute this from first
% principles; instead you:
%   - ask the TPCT vendor for a performance curve, OR
%   - use a conservative internal boiling HTC range.
%
% Here we keep a conservative value and clearly mark it as "TPCT vendor input".
h_boil = 12000;                    % [W/m^2-K] NEED TPCT VENDOR DATA (placeholder)
R_boil = 1/(h_boil*Aev_id);        % [K/W] (CALC)

% Total evaporator-side resistance from steam to TPCT working fluid:
R_evap_total = R_steam + R_wall_ev + R_boil;  % [K/W] (CALC)

% Allowable evaporator temperature drop (design criterion)
dT_allow_evap = 12;                % [K] TEAM DESIGN CRITERION

%% ========================================================================
% 6) AIR-COOLED CONDENSER (PREFERRED: VENDOR MODULE UA)
% ========================================================================
% Because coil internal geometry is often proprietary, the most defensible
% approach is to use *vendor module surface area and overall U* we have.
% This keeps the theory correct (Q = UA * F * LMTD) while avoiding
% reverse-engineering fin pitch/tube pitch.
%
% Vendor module used here: Kelvion RF-NC101E4H drycooler (example)
% - Surface area: 197.1 m^2  (spec)
% - Airflow: 19,400 m^3/h    (spec)
% - Dimensions: 2220x1610x1350 mm (L x W x H) (listing)
% Sources:
%   https://www.hosbv.com/data/specifications/15917_15917_Kelvion_RF-
    NC101E4H_Drycooler.pdf
%   https://hosbv.com/en/product/17499/condensers/Kelvion-RF-NC101E4H-.html

coil.vendor = 'Kelvion RF-NC101E4H (example drycooler module)';
coil.Ao_total_per_module = 197.1;            % [m^2] external surface area (
    Kelvion spec)
coil.airflow_m3ph = 19400;                   % [m^3/h] airflow (Kelvion spec)

% Convert airflow to SI volumetric flow:
coil.Vdot_air_mod = coil.airflow_m3ph/3600;  % [m^3/s] (CALC)

% Module footprint / frontal dimensions:
% From listing: sizes 2220 x 1610 x 1350 mm (L x W x H). We treat:
%   W = 1.610 m, H = 1.350 m as the frontal area (air passes through W H).
coil.mod_W = 1.610;                          % [m] module width (Kelvion listing)
coil.mod_H = 1.350;                          % [m] module height (Kelvion listing
    )
coil.A_frontal = coil.mod_W*coil.mod_H;      % [m^2] (CALC)

% Face velocity implied by vendor airflow (useful sanity check):
V_face_vendor = coil.Vdot_air_mod/coil.A_frontal; % [m/s] (CALC)

% Air energy capacity per module (simple energy balance):
% Q = m_dot_air * cp * dT_air
mdot_air_mod = rho_air*coil.Vdot_air_mod;          % [kg/s] (CALC)
Qcap_mod = mdot_air_mod*cp_air*dT_air;             % [W] (CALC)

```

```
% Apply crosswind multiplier (if crosswind increases required flow / modules)
Qcap_mod_effective = Qcap_mod/crosswind_multiplier; % [W] (TEAM SLIDES multiplier)

% Modules needed by *air energy balance*:
N_modules = max(1, ceil(Q_total/Qcap_mod_effective)); % [-] integer count (CALC)

% Total airflow and estimated fan power (using your slide dP and eta_fan):
Vdot_air_total = coil.Vdot_air_mod*N_modules;          % [m^3/s] (CALC)
P_fan_total = dP_air*Vdot_air_total/eta_fan;           % [W] (CALC)

% Now check UA requirement using LMTD:
T_air_out_C = T_air_in_C + dT_air;                     % [ C ] (CALC)
DT1 = T_cond_surface_C - T_air_in_C;                   % [K] (CALC)
DT2 = T_cond_surface_C - T_air_out_C;                  % [K] (CALC)

% LMTD for a constant-temperature condensing surface vs air warming:
DTlm = (DT1 - DT2)/log(DT1/DT2);                       % [K] (CALC)

UA_req = Q_total/(F_LMTD*DTlm);                        % [W/K] required UA (TEAM F_LMTD)

% Available UA from modules:
UA_avail = U_overall * coil.Ao_total_per_module * N_modules; % [W/K] (TEAM U_overall + vendor Ao)

UA_margin = UA_avail/UA_req;                           % [-] >1 means enough UA

%% ========================================================================
% 7) TPCT HEAT-TRANSPORT LIMITS (FLOODING + BOILING + SONIC CHECK)
% ========================================================================
% Even if the air cooler has enough UA, each TPCT can only carry so much
% heat before hitting internal limits. For wickless thermosyphons, flooding
% / entrainment is often dominant at high powers.
%
% We keep the same functional forms as your code (engineering correlation).
%
% Representative TPCT operating temperature:
T_tp_C = (Tsat_C + T_cond_surface_C)/2;                % [ C ] (TEAM ASSUMPTION)
wt = satWater_simple(T_tp_C);                          % sat properties (internal)

% Surface tension of water from IAPWS 2014 (official)
sigma_w = surfaceTension_IAPWS2014(T_tp_C);            % [N/m] (IAPWS 2014)

A_v = pi*(tp.ID^2)/4;                                  % [m^2] vapor core area (CALC)

% Flooding limit correlation (engineering form)
Qmax_flood = QmaxFlood_FaghriStyle(tp.ID, A_v, wt.rho_l, wt.rho_v, h_fg, sigma_w, g); % (CORRELATION)
Qmax_design = 0.60*Qmax_flood;                         % [W] design margin factor (TEAM CHOICE)

% Boiling/CHF margin using Zuber pool boiling CHF (very conservative bound)
qCHF_frac = 0.30;                                      % [-] design criterion (TEAM)
```

```
qCHF_Zuber = CHF_ZuberPoolBoiling(wt.rho_l, wt.rho_v, h_fg, sigma_w, g); % [W/m^2] (CORRELATION)
Qmax_boil = qCHF_frac*qCHF_Zuber * (pi*tp.ID*tp.Lev);     % [W] (CALC)

% Sonic limit check (rough):
% Speed of sound for steam ~400 m/s order at these conditions. This should
% be replaced by a property call if you need tight accuracy.
a_v = 430;                                                % [m/s] TEAM ASSUMPTION
G_sonic_allow = 0.25*wt.rho_v*a_v;                         % [kg/m^2/s] conservative fraction (TEAM)
Qmax_sonic = G_sonic_allow*A_v*h_fg;                       % [W] (CALC)

% TPCT maximum per-pipe heat transport:
Qmax_tpct = min([Qmax_design, Qmax_boil, Qmax_sonic]);     % [W] (CALC)

%% ========================================================================
% 8) NUMBER OF THERMOSYPHONS REQUIRED (BASED ON LIMITS + TEMPERATURE DROP)
% ========================================================================
% We size N_pipes so that:
%   (1) each pipe carries less than Qmax_tpct, AND
%   (2) evaporator-side temperature drop is within dT_allow_evap.
%
% Condenser-side resistances:
% Internal condensation HTC in TPCT condenser is also a vendor input.
h_cond_int = 12000;                                       % [W/m^2-K] NEED TPCT VENDOR DATA
R_cond_int = 1/(h_cond_int*Aco_id);                        % [K/W] (CALC)

% Air-side resistance per pipe is taken from the *module UA* approach:
% We assign each pipe an equal share of total UA. This is a simplification,
% but good for preliminary sizing when pipes are evenly distributed.
%
% Total air-side resistance = 1 / UA_avail_total
R_air_total = 1/(UA_avail);                                % [K/W] (CALC)

% Now find N_pipes with an adaptive loop:
N_pipes = 5000;                                           % [-] initial guess (TEAM GUESS)
while true
    Q_pipe = Q_total/N_pipes;                              % [W] per-pipe duty (CALC)

    % Evaporator temperature drop:
    dT_evap = Q_pipe*R_evap_total;                         % [K] (CALC)

    % Condenser temperature drop per pipe:
    % Each pipe shares air-side UA equally, so air-side R per pipe scales with N_pipes.
    R_air_per_pipe = R_air_total * N_pipes;                % [K/W] (CALC)
    R_cond_total = R_cond_int + R_wall_co + R_air_per_pipe;% [K/W] (CALC)
    dT_cond = Q_pipe*R_cond_total;                         % [K] (CALC)

    limits_ok = (Q_pipe <= Qmax_tpct);
    dT_ok     = (dT_evap <= dT_allow_evap);

    if limits_ok && dT_ok
        break;
```

```
    end

    % Increase pipe count; larger steps if limit violation is big
    if ~limits_ok
        N_pipes = N_pipes + 2000;
    else
        N_pipes = N_pipes + 500;
    end
end

Q_pipe = Q_total/N_pipes;                                   % [W] (CALC)
dT_evap = Q_pipe*R_evap_total;                              % [K] (CALC)
dT_cond = Q_pipe*R_cond_total;                              % [K] (CALC)
dT_total_pipe = dT_evap + dT_cond;                           % [K] (CALC)

%% ========================================================================
% 9) CONDENSATE DRAINAGE CHECK IN POROUS RECEIVER (DARCY)
% ========================================================================
% Steam condensate produced is approximately the incoming vapor mass flow.
m_dot_cond_total = m_dot_vapor;                             % [kg/s] (CALC)

% Condensate per pipe:
m_dot_cond_per_pipe = m_dot_cond_total/N_pipes;             % [kg/s] (CALC)
Vdot_cond_per_pipe = m_dot_cond_per_pipe / w.rho_l;         % [m^3/s] (CALC)

% Darcy superficial velocity through foam annulus:
v_Darcy = Vdot_cond_per_pipe / A_cs_foam;                   % [m/s] (CALC)

% Darcy pressure drop required: dP/dz = mu * v / K
dP_dz = mu_l * v_Darcy / foam.K_perm;                       % [Pa/m] (CALC)
dP_total = dP_dz * tp.Lev;                                  % [Pa] (CALC)

% Available hydrostatic head (liquid column height ~ Lev):
dP_head = w.rho_l*g*tp.Lev;                                 % [Pa] (CALC)

drain_ok = (dP_total <= 0.5*dP_head);                       % [-] 50% margin (
    TEAM CRITERION)

%% ========================================================================
% 10) REPORT (HUMAN-READABLE OUTPUT)
% ========================================================================
fprintf('\n==================== DESIGN REPORT (VENDOR-AWARE)
    ====================\n');
fprintf('TOTAL DUTY: Q_total = %.2f MW\n', Q_total/1e6);

fprintf('\n--- Steam side ---\n');
fprintf('Steam Tsat=%.1f C (Psat~%.2f bar), x_in=%.2f\n', Tsat_C, Psat_bar, x_in)
    ;
fprintf('Mixture flow m_dot_mix = %.2f kg/s; vapor to condense m_dot_vapor = %.2f
     kg/s\n', m_dot_mix, m_dot_vapor);

fprintf('\n--- Condensation on receiver ---\n');
fprintf('Assumed  T (sat-to-surface)=%.1f K\n', dT_steam_receiver);
fprintf('Nusselt film h_nusselt=%.0f W/m^2K; foam enhancement F=%.2f => h_steam
    =%.0f W/m^2K\n', h_nusselt, foam.F_cond_enh, h_steam);
fprintf('Foam fin efficiency eta_foam=%.3f; A_eff_steam per TPCT=%.3f m^2\n',
    eta_foam, A_eff_steam);
```

```
519
520 fprintf('\n--- Air cooler modules (Kelvion example) ---\n');
521 fprintf('Vendor module: %s\n', coil.vendor);
522 fprintf('Ao per module=%.1f m^2; airflow per module=%.1f m^3/s; implied face V
    =%.2f m/s\n', ...
523     coil.Ao_total_per_module, coil.Vdot_air_mod, V_face_vendor);
524 fprintf('Air energy per module Qcap_mod=%.1f kW (before crosswind factor)\n',
    Qcap_mod/1e3);
525 fprintf('Crosswind multiplier=%.2f => effective Qcap_mod=%.1f kW\n',
    crosswind_multiplier, Qcap_mod_effective/1e3);
526 fprintf('Modules by air energy balance: N_modules=%d\n', N_modules);
527 fprintf('Total airflow=%.1f m^3/s; fan power %.2f MW ( P =%.0f Pa,   =%.2f)\n',
     Vdot_air_total, P_fan_total/1e6, dP_air, eta_fan);
528
529 fprintf('\n--- UA check (vendor surface + slide U_overall) ---\n');
530 fprintf('T_cond_surface=%.1f C, air in=%.1f C, air out=%.1f C => LMTD=%.2f K\n',
    ...
531     T_cond_surface_C, T_air_in_C, T_air_out_C, DTlm);
532 fprintf('UA required = %.3e W/K (includes F_LMTD=%.2f)\n', UA_req, F_LMTD);
533 fprintf('UA available = %.3e W/K (U_overall=%.1f, Ao_total=%g m^2, modules=%d)\n'
    , ...
534     UA_avail, U_overall, coil.Ao_total_per_module, N_modules);
535 fprintf('UA margin (avail/req) = %.2f\n', UA_margin);
536
537 fprintf('\n--- TPCT per-pipe limits ---\n');
538 fprintf('Qmax_flood*0.60=%.2f kW, Qmax_boil=%.2f kW, Qmax_sonic=%.2f kW =>
    Qmax_tpct=%.2f kW\n', ...
539     Qmax_design/1e3, Qmax_boil/1e3, Qmax_sonic/1e3, Qmax_tpct/1e3);
540
541 fprintf('\n--- Selected TPCT count (preliminary) ---\n');
542 fprintf('N_pipes=%d => Q_pipe=%.2f kW\n', N_pipes, Q_pipe/1e3);
543 fprintf('Per-pipe  T_evap =%.2f K (allow %.1f K),  T_cond =%.2f K, total   T
    %.2f K\n', ...
544     dT_evap, dT_allow_evap, dT_cond, dT_total_pipe);
545
546 fprintf('\n--- Condensate drainage (Darcy) ---\n');
547 fprintf('Per-pipe condensate Vdot=%.3e m^3/s; Darcy v=%.3e m/s\n',
    Vdot_cond_per_pipe, v_Darcy);
548 fprintf('Required  P  across Lev: %.0f Pa; available hydrostatic head: %.0f Pa =>
     drain_ok=%d\n', ...
549     dP_total, dP_head, drain_ok);
550
551 fprintf('=========================================================================\
    n');
552
553 %% =========================================================================
554
555 % -------- PACK OUTPUTS --------
556 out = struct();
557 out.Q_total_W = Q_total;
558 if exist('N_modules','var'); out.N_modules = N_modules; end
559 if exist('P_fan_total','var'); out.P_fan_total_W = P_fan_total; end
560 if exist('Vdot_air_total','var'); out.Vdot_air_total_m3s = Vdot_air_total; end
561 if exist('mdot_air_total','var'); out.mdot_air_total_kgs = mdot_air_total; end
562 if exist('T_air_out_C','var'); out.T_air_out_C = T_air_out_C; end
563 if exist('UA_req','var'); out.UA_req_WK = UA_req; end
564 if exist('UA_avail','var'); out.UA_avail_WK = UA_avail; end
```

```
if exist('UA_margin','var'); out.UA_margin = UA_margin; end
if exist('Qcap_mod_effective','var'); out.Qcap_mod_effective_W = Qcap_mod_effective; end
if exist('coil','var')
    if isfield(coil,'Ao_total_per_module'); out.Ao_total_per_module_m2 = coil.Ao_total_per_module; end
    if isfield(coil,'Vdot_air_mod'); out.Vdot_air_mod_m3s = coil.Vdot_air_mod; end
    if isfield(coil,'A_frontal'); out.A_frontal_m2 = coil.A_frontal; end
    if isfield(coil,'mod_W'); out.mod_W_m = coil.mod_W; end
    if isfield(coil,'mod_H'); out.mod_H_m = coil.mod_H; end
end
if exist('N_TPCT_total','var'); out.N_TPCT_total = N_TPCT_total; end
if exist('Qmax_TPCT_flood_W','var'); out.Qmax_TPCT_flood_W = Qmax_TPCT_flood_W; end
if exist('qpp_evap','var'); out.qpp_evap_Wm2 = qpp_evap; end
if exist('qpp_CHF','var'); out.qpp_CHF_Wm2 = qpp_CHF; end

end

% ========================================================================
% ORIGINAL SUBFUNCTIONS FROM MODEL_2 (unchanged)
% ========================================================================
function st = satWater_simple(T_C)
% satWater_simple
%   Quick saturated-water lookup for 50   100C   , linear interpolation.
%   Returns:
%     psat [Pa], rho_l [kg/m3], rho_v [kg/m3], hf [J/kg], hfg [J/kg]
%
%   Source of numbers: typical steam tables (embedded dataset).
%   Upgrade path: replace with IF97 (IAPWS) or CoolProp for production work.

T = [50 55 60 65 70 75 80 85 90 95 100];
ps_MPa = [0.012352 0.015761 0.019946 0.025042 0.031201 0.038563 0.047373 0.057834 0.070140 0.084552 0.101325];
rho_l  = [988.05 985.65 983.16 980.52 977.76 974.89 971.91 968.82 965.62 962.33 958.37];
rho_v  = [0.08302 0.10266 0.13043 0.16146 0.19833 0.24418 0.29216 0.34569 0.41451 0.49220 0.59752];
hf_kJ  = [209.33 230.23 251.18 272.12 292.98 313.93 334.91 355.90 376.92 397.96 419.04];
hfg_kJ = [2382.7 2370.7 2357.7 2345.4 2333.8 2321.4 2308.8 2296.0 2283.2 2270.2 2257.0];

T_C = max(min(T_C, max(T)), min(T));

st.psat = interp1(T, ps_MPa, T_C, 'linear')*1e6;
st.rho_l = interp1(T, rho_l,  T_C, 'linear');
st.rho_v = interp1(T, rho_v,  T_C, 'linear');
st.hf  = interp1(T, hf_kJ,  T_C, 'linear')*1e3;
st.hfg = interp1(T, hfg_kJ, T_C, 'linear')*1e3;
end

function mu = muWater_simple(T_C)
% muWater_simple
%   Rough liquid water viscosity correlation (Andrade-type).
%   Valid-ish for ~0   100C   . Replace with IAPWS viscosity for higher fidelity.
```

```
T = T_C + 273.15;
mu = 2.414e-5*10^(247.8/(T-140));
end

function k = kWater_simple(T_C)
% kWater_simple
%   Rough liquid water thermal conductivity fit, 0   100C   .
k = 0.561 + 0.0019*T_C - 1.0e-5*T_C.^2;
end

function h = hFilmCond_NusseltVertical(rhoL, rhoV, muL, kL, hfg, g, L, dT)
% hFilmCond_NusseltVertical
%   Nusselt laminar film condensation on a vertical surface (average h).
%   h = 0.943 * [ rhoL*(rhoL-rhoV)*g*hfg*kL^3 / (muL*L*dT) ]^(1/4)
h = 0.943 * (rhoL*(rhoL-rhoV)*g*hfg*kL^3/(muL*L*max(dT,1e-6)))^(1/4);
end

function sigma = surfaceTension_IAPWS2014(T_C)
% surfaceTension_IAPWS2014
%   Official IAPWS (2014) surface tension correlation:
%     sigma = B * tau^mu * (1 + b*tau),  tau = 1 - T/Tc
%   Parameters from IAPWS PDF:
%     Tc=647.096 K, B=235.8 mN/m, mu=1.256, b=-0.625
%   Source: https://iapws.org/public/documents/CH-L9/Surf-H2O-2014.pdf
T = T_C + 273.15;
Tc = 647.096;
tau = 1 - T/Tc;
B = 235.8e-3;  mu = 1.256;  b = -0.625;
sigma = B*(tau^mu)*(1 + b*tau);
end

function Qmax = QmaxFlood_FaghriStyle(D, Av, rhoL, rhoV, hfg, sigma, g)
% QmaxFlood_FaghriStyle
%   Engineering flooding/entrainment limit form for wickless thermosyphons.
%   Used as a practical bound; vendor data should replace if available.
Bo = sqrt((rhoL - rhoV)*g*D^2/sigma);
K  = (rhoL/rhoV)^0.14 * (tanh(sqrt(Bo)))^2;
term1 = (g*sigma*(rhoL - rhoV))^(0.25);
term2 = (rhoV^(-0.5) + rhoL^(-0.5))^(-2);
Qmax = K*hfg*Av*term1*term2;
end

function qCHF = CHF_ZuberPoolBoiling(rhoL, rhoV, hfg, sigma, g)
% CHF_ZuberPoolBoiling
%   Zuber CHF correlation for pool boiling (large horizontal surface):
%   q''_CHF = 0.131 * hfg * rhoV^(1/2) * [ sigma*g*(rhoL-rhoV) ]^(1/4)
qCHF = 0.131 * hfg * sqrt(rhoV) * (sigma*g*(rhoL-rhoV))^(1/4);
end
```

Listing 7: HR porous Code